基于遗传算法和粒子群算法的烟幕干扰弹投放策略优化

摘要

本文针对无人机烟幕对抗来袭导弹问题，基于几何分析建立时空动力学模型，采用遗传算法与粒子群优化算法，分别求解单机与多机协同场景下的最优干扰策略，以实现总遮蔽时长的最大化。

针对问题一，本文严格判断遮蔽条件，且选取圆柱体（真目标）的很多重要轮廓点来离散化处理目标点, 建立以最大遮蔽时长为目标函数的几何光学模型。由于只要考虑一个烟幕，本文选择使用二分搜索临界时间求解法，通过二分法计算出遮蔽发生的起始时间和终止时间，进而得到完整的遮蔽时间。最终得到在给定参数条件下，烟幕弹对导弹攻击的有效遮蔽发生在 8.06s-9.45s 时间段内，**最大遮蔽时长为：1.39 秒**。

针对问题二，本文以无人机飞行方位角、飞行速度、烟幕弹投放时间和起爆延迟作为决策变量，建立以遮蔽时长最大化为目标函数的多参数优化模型。本文选择使用遗传算法 [1] 进行全局寻优，得到粗略结果后，使用二分精确搜寻，通过 50 代进化过程，得到粗略结果后经过进一步优化，最终得到最优参数组合：飞行方位角约 0.25 度，飞行速度 103.79 m/s，投放时间 0.52s，起爆延迟 0.53s，对应的**最大遮蔽时长为：4.59 秒**。

针对问题三，本文建立 8 维参数空间的非线性约束优化模型，采用粒子群优化算法求解。通过 300 个粒子、50 代进化的优化过程，获得最优投放策略：无人机以 108.43 m/s 速度、8.36° 方向飞行，分别在 0s、1.00s、9.10s 投放三枚烟幕弹，对应起爆延迟为 0s、0s、0.35s。优化结果显示**总遮蔽时长为 6.41 秒**。

针对问题四，本文建立 12 维参数空间的非线性约束优化模型，采用改进的粒子群优化算法求解。通过多代进化的深度优化过程，获得最优协同策略：FY1 以 126.11 m/s 速度、7.38° 方向在 0.23s 投放，FY2 以 118.2 m/s 速度、319.85° 方向在 15.00s 投放，FY3 以 116.33m/s 速度、72.15° 方向在 27.19s 投放，对应起爆延迟为 0.36s、2.38s、0.36s。优化结果显示**总遮蔽时长为 11.425 秒**。

针对问题五，本文在考虑多烟幕协同作用 [2] 的基础上建立 40 维参数空间的大规模非线性约束优化模型，使用深度改进粒子群优化算法 [3] 进行全局寻优，采用并行计算架构和缓存机制，通过 300 个粒子、200 代进化的大规模优化过程，获得最优集群协同策略和**总遮蔽时长为 22.61 秒**。

关键词：干扰弹投放策略优化　几何光学　遗传算法　粒子群算法

1 问题重述

1.1 问题背景

烟幕干扰弹通过化学燃烧形成遮蔽云团，在导弹与目标间建立屏障以保护真实目标。无人机作为投放平台，需要在复杂的三维空间中精确控制投放时机和位置，以实现最佳干扰效果。本问题涉及 3 枚来袭导弹（M1、M2、M3）和 5 架无人机（FY1-FY5）的协同作战场景。

来袭导弹为空地导弹，飞行速度 300 m/s，直指假目标 (0,0,0)。真目标为半径 7m、高 10m 的圆柱体，底面圆心位于 (0,200,0)。无人机可携带烟幕干扰弹，速度范围 70-140 m/s，投放间隔 ≥ 1s。烟幕弹起爆后形成半径 10m 的球状云团，有效时间 20s，下沉速度 3 m/s。

1.2 问题提出

问题一：分析无人机 FY1 以固定参数（120 m/s 朝假目标方向，1.5s 后投放，3.6s 延迟起爆）投放烟幕弹对导弹 M1 的遮蔽效果，计算有效遮蔽时长。

问题二：优化无人机 FY1 的飞行参数和投放策略，使对导弹 M1 的遮蔽时间最大化。

问题三：设计无人机 FY1 投放 3 枚烟幕弹的序贯策略，实现对导弹 M1 的持续遮蔽。

问题四：利用 FY1、FY2、FY3 等 3 架无人机，各投放 1 枚烟幕干扰弹，实施对 M1 的干扰，设计多机协同投放策略。

问题五：利用 5 架无人机，每架无人机至多投放 3 枚烟幕干扰弹，实施对 M1、M2、M3 等 3 枚来袭导弹的干扰，设计大规模集群协同投放策略。

2 问题分析

问题一：问题一需要在无人机飞行参数、投放时机、起爆延时等关键参数固定的情况下，计算烟幕干扰弹对导弹 M1 的有效遮蔽时长。我们为了准确计算遮蔽效果，建立空间了坐标系模型和运动轨迹模型，分析导弹、无人机、烟幕弹的时空关系，最后以遮蔽时间作为评价指标，建立基于几何遮蔽原理的物理仿真模型。

问题二：问题二需要在四维参数空间中优化无人机飞行方位角、飞行速度、烟幕弹投放时间和起爆延迟等关键决策变量，使得烟幕干扰弹对导弹 M1 的有效遮蔽时长达到最大化。我们为了准确求解这一优化问题，建立了多变量耦合的目标函数模型和约束条件模型，分析参数变化对遮蔽效果的非线性影响规律，最后以遮蔽时长最大化为优化目标，建立基于智能搜索算法的全局优化模型。

问题三：问题三需要在 8 维参数空间中优化单架无人机的飞行方位角、飞行速度以及 3 枚烟幕弹的投放时间和起爆延迟等关键决策变量，使得多枚烟幕干扰弹对导弹 M1 的协同遮蔽时长达到最大化。我们为了准确求解这一多弹协同优化问题，建立了时序约

束模型和多云团协同遮蔽效果模型，分析投放间隔约束对遮蔽连续性的影响规律以及多个烟幕云团的空间-时间协同机制，最后以协同遮蔽时长最大化为优化目标，建立基于粒子群优化算法的高维约束优化模型。

问题四：问题四需要在 12 维参数空间中优化 3 架无人机（FY1、FY2、FY3）的飞行方位角、飞行速度、烟幕弹投放时间和起爆延迟等关键决策变量，使得多机协同投放烟幕干扰弹对导弹 M1 的综合遮蔽时长达到最大化。我们为了准确求解这一大规模多智能体协同优化问题，建立了异构初始条件模型和多机空间-时序协调模型，分析不同无人机间的协同效应机制以及空间分布对遮蔽覆盖范围的影响规律，最后以多机协同遮蔽时长最大化为优化目标，建立基于改进粒子群优化算法的大规模约束优化模型。

问题五：问题五需要在 40 维超高维参数空间中优化 5 架无人机（FY1-FY5）的飞行方位角、飞行速度以及每架无人机至多 3 枚烟幕弹的投放时间和起爆延迟等关键决策变量，使得大规模集群协同投放烟幕干扰弹对 3 枚来袭导弹（M1、M2、M3）的综合干扰时长达到最大化。为了准确求解这一极大规模多智能体多目标协同优化问题，我们需要建立多导弹威胁评估模型和集群资源分配优化模型，分析多机集群间的复杂协同效应机制以及针对多目标威胁的最优资源分配策略，最后以多导弹总干扰时长最大化为优化目标，建立基于深度改进粒子群优化算法的超大规模约束多目标优化模型。

3 模型假设与符号说明

3.1 模型基本假设

- 导弹以恒定速度沿直线飞行，可视为质点。
- 多架无人机之间无碰撞风险，可视为质点。
- 烟幕弹脱离无人机后仅受重力作用，空气阻力忽略不计。
- 烟幕云团形状为规则球体，遮蔽效果在有效范围内均匀分布。
- 风速、湿度等环境因素的影响忽略不计。

3.2 符号说明

符号	含义	单位
$\vec{r}_{M_i}$	第 i 枚导弹位置坐标	m
$\vec{r}_{FY_j}$	第 j 架无人机位置坐标	m
v_m	导弹飞行速度	m/s
v_{FY_j}	第 j 架无人机飞行速度	m/s
$t_{release}$	烟幕弹投放时间	s
$t_{explosion}$	烟幕弹起爆时间	s
R_{smoke}	烟幕有效遮蔽半径	m
T_{last}	烟幕有效遮蔽持续时间	s
v_{sink}	烟幕云团下沉速度	m/s
θ_j	第 j 架无人机飞行方向角	rad
$\vec{r}_{release}$	烟幕弹投放点坐标	m
$\vec{r}_{explosion}$	烟幕弹起爆点坐标	m
T_{shield}	有效遮蔽时间	s
d_{min}	导弹轨迹与烟幕中心距离	m
g	重力加速度	m/s^2
α_{coop}	协同效应系数	-

4 模型建立与求解

4.1 问题一模型建立与求解

4.1.1 问题一求解思路

烟幕弹干扰导弹攻击问题的核心在于分析烟幕云团在时空中的分布特性以及其对导弹视线的遮蔽效果。我们决定采用几何光学原理和三维空间解析几何方法，建立动态时空模型来求解烟幕的有效遮蔽时间。

求解思路：

1. 建立三维坐标系：以目标区域为原点，建立空间直角坐标系，描述导弹、无人机、烟幕弹和目标的空间位置关系。

2. 构建运动轨迹模型：基于物理运动规律，分别建立导弹匀速直线运动模型、无人机匀速飞行模型和烟幕弹抛物线运动模型。

3. 建立遮蔽判断机制：采用点到直线距离算法，判断烟幕云团是否有效遮蔽导弹对目标的视线 [4]。

4.1.2 问题一模型建立

1. 坐标系建立与参数定义

建立三维直角坐标系 O-xyz，其中原点 O 设置在假目标位置 (0,0,0)，z 轴垂直向上，xy 平面为水平面。真目标为圆柱体，底面圆心坐标为 $(0, 200, 0)$，半径 $r_{target} = 7\text{m}$，高度 $h_{target} = 10\text{m}$。

主要参数设置：

- 导弹初始位置：$(20000, 0, 2000)$
- 导弹速度：$v_{missile} = 300$ m/s
- 无人机初始位置：$(17800, 0, 1800)$
- 无人机速度：$v_{plane} = 120$ m/s
- 烟幕云团半径：$R_{smoke} = 10$ m
- 烟幕有效持续时间：$T_{last} = 20$ s
- 投放时间：$t_{release} = 1.5$ s
- 起爆延迟：$\tau = 3.6$ s

2. 运动轨迹数学模型

（1）导弹运动模型

导弹做匀速直线运动，目标点为 $(0, 0, 0)$：

$$\vec{r}_{missile}(t) = \vec{r}_{missile}^{(0)} + \vec{v}_{missile} \cdot t \tag{1}$$

其中：

$$\vec{v}_{missile} = v_{missile} \cdot \frac{\vec{r}_{target} - \vec{r}_{missile}^{(0)}}{|\vec{r}_{target} - \vec{r}_{missile}^{(0)}|} \tag{2}$$

（2）无人机运动模型

无人机朝假目标方向做匀速直线飞行：

$$\vec{r}_{plane}(t) = \vec{r}_{plane}^{(0)} + \vec{v}_{plane} \cdot t \tag{3}$$

飞行方向向量：

$$\vec{v}_{plane} = v_{plane} \cdot \frac{(0, 0, 0) - \vec{r}_{plane}^{(0)}}{|(0, 0, 0) - \vec{r}_{plane}^{(0)}|} \tag{4}$$

（3）烟幕弹运动模型

烟幕弹在 $t = t_{release} = 1.5$s 时投放，经过 $\tau = 3.6$s 后爆炸：

投放位置：

$$\vec{r}_{release} = \vec{r}_{plane}(t_{release}) \tag{5}$$

爆炸位置：

$$\vec{r}_{explosion} = \vec{r}_{release} + \vec{v}_{plane} \cdot \tau + (0, 0, -\frac{1}{2}g\tau^2) \tag{6}$$

爆炸时间：

$$t_{explosion} = t_{release} + \tau = 5.1\text{s} \tag{7}$$

（4）烟幕云团位置模型

烟幕云团以速度 $v_{sink} = 3$ m/s 匀速下沉：

$$\vec{r}_{cloud}(t) = \begin{cases} \text{null}, & t < t_{explosion} \\ \vec{r}_{explosion} + (0, 0, -v_{sink}(t - t_{explosion})), & t_{explosion} \leq t \leq t_{explosion} + T_{last} \\ \text{invalid}, & t > t_{explosion} + T_{last} \end{cases} \tag{8}$$

3. 遮蔽效果判断模型

（1）时间有效性判断

烟幕云团的时间有效性：

$$t_{explosion} \leq t \leq t_{explosion} + T_{last} \tag{9}$$

（2）空间遮蔽判断

我们通过采取对目标采样代替整个目标的方法，由于目标是圆柱体，只需要在上下底面采样就能完整替代整个圆柱体，进一步优化数值计算判断的效率。

对于导弹位置 $\vec{M}$ 和目标采样点 $\vec{P}$，视线向量为：

$$\vec{L} = \vec{P} - \vec{M} \tag{10}$$

云团中心 $\vec{C}$ 到视线的最短距离计算：

投影参数：

$$t_{proj} = \frac{(\vec{C} - \vec{M}) \cdot \vec{L}}{|\vec{L}|^2} \tag{11}$$

最短距离：

$$d_{min} = \left| \vec{C} - \vec{M} - t_{proj} \cdot \vec{L} \right| \tag{12}$$

遮蔽条件：

$$\text{遮蔽有效} = \begin{cases} \text{True}, & d_{min} < R_{smoke} \text{ 且 } 0 \leq t_{proj} \leq 1 \\ \text{True}, & t_{proj} < 0 \text{ 且 } |\vec{C} - \vec{M}| < R_{smoke} \\ \text{True}, & t_{proj} > 1 \text{ 且 } |\vec{C} - \vec{P}| < R_{smoke} \\ \text{False}, & \text{否则} \end{cases} \tag{13}$$

4.1.3 问题一模型求解与分析

采用 0.1s 的时间粗网格预先搜索 5.1 到 25.1s 内的遮蔽情况，在零界点用二分搜索得到精确的遮蔽起始点和遮蔽终点

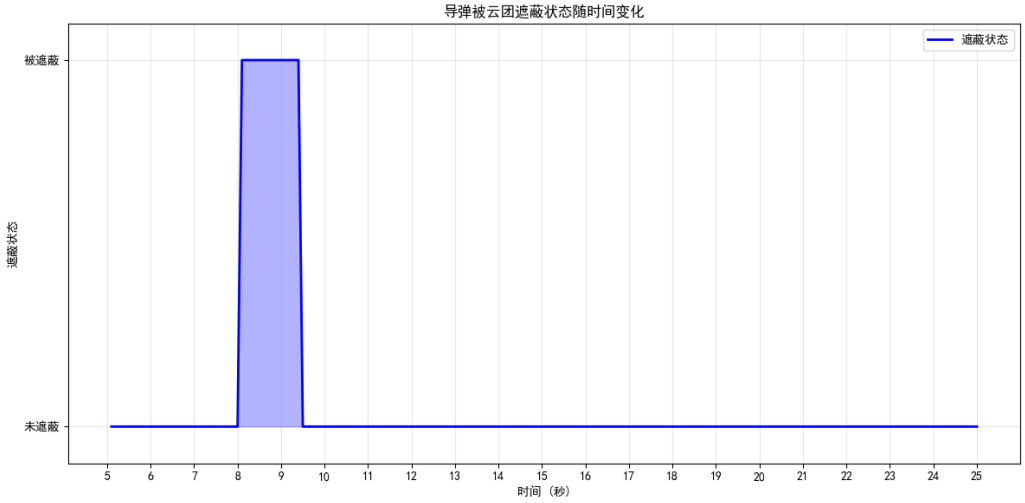


图 1: 5.1 到 25.1s 内真目标的遮蔽情况

通过数值计算得到：

参数类别	具体内容
爆炸时间	5.1 s
爆炸位置	$(17188.0, 0.0, 1736.5)$ m
遮蔽时间段	8.06 s - 9.45 s（持续 1.39 s）
最大遮蔽时长	**1.3900 秒**

在给定条件下，烟幕弹能够对导弹攻击产生有效干扰，最大遮蔽时长为 1.3900 秒。

4.2 问题二模型建立与求解

4.2.1 问题二求解思路

问题二的本质是一个多维黑盒优化问题，目标是在给定的参数空间内寻找使遮蔽时长最大化的参数组合。我们考虑到问题的非线性特性和可能存在的多峰值特征，选择遗传算法作为求解方法。

4.2.2 问题二模型建立

1. 优化模型数学表述

决策变量：

$$\mathbf{X} = [\theta, v, t_{release}, \tau]^T \tag{14}$$

目标函数：

$$\max f(\theta, v, t_{release}, \tau) = T_{shield}(\theta, v, t_{release}, \tau) \tag{15}$$

约束条件：

$$\begin{cases} -\frac{\pi}{10} \leq \theta \leq \frac{\pi}{10} \\ 70 \leq v \leq 140 \\ 0 \leq t_{release} \leq 4 \\ 0 \leq \tau \leq 1 \\ z_{direction} = 0 \text{ (水平飞行约束)} \end{cases} \tag{16}$$

2. 遗传算法模型设计

（1）个体编码

采用实数编码方式：

$$\text{Individual} = [\theta, v, t_{release}, \tau] \tag{17}$$

（2）适应度函数

$$f(\mathbf{x}) = \text{calculate_shielding_duration}(\theta, v, t_{release}, \tau) \tag{18}$$

其中 $\mathbf{x} = [\theta, v, t_{release}, \tau]^T$ 为决策变量向量。

（3）遗传操作

选择算子：锦标赛选择，锦标赛规模为 3

交叉算子：混合交叉（Blend Crossover）

$$\text{child}_i = \text{parent}_1 + \alpha \cdot (\text{parent}_2 - \text{parent}_1) \tag{19}$$

其中 $\alpha = 0.5$。

变异算子：高斯变异

$$\text{gene}_{new} = \text{gene}_{old} + N(0, \sigma^2) \tag{20}$$

变异标准差：$\sigma = [0.2, 5, 0.2, 0.2]$，变异概率 $p_m = 0.2$。

4.2.3 问题二模型求解与分析

算法参数设置：种群规模 50 个个体，进化代数 50 代，交叉概率 0.7，变异概率 0.2。

最优解分析：

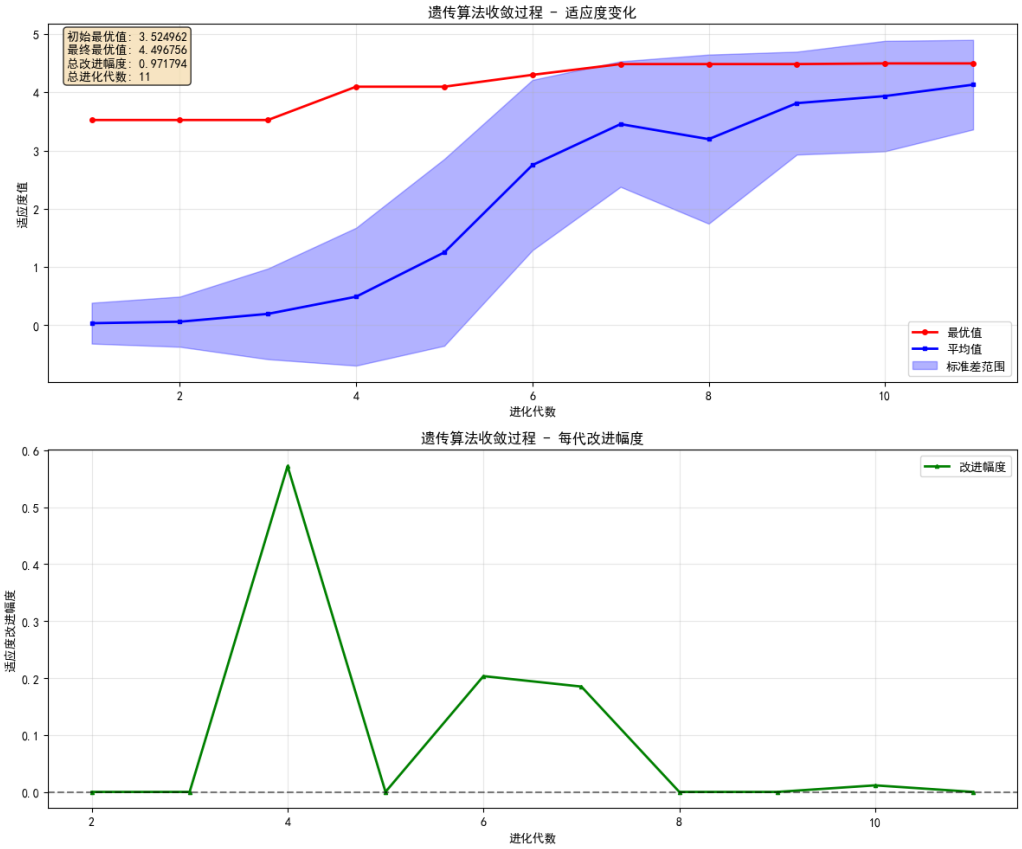


图 2: 遗传算法迭代情况

参数名称	**参数值**
飞行方位角	约 0.25°
飞行速度	103.79 m/s
投放时间	0.52 s
起爆延迟	0.53 s
投放点坐标	$(17842.96, 4.46, 1800.00)$ m
起爆点坐标	$(17906.50, 11.04, 1798.60)$ m
最大遮蔽时长	**4.5900 秒**

通过遗传算法优化，我们确定了烟幕弹干扰的最优参数配置，在该配置下最大遮蔽时长为 4.5900 秒。

4.3 问题三模型建立与求解

4.3.1 问题三求解思路

问题三，四，五目标函数的自变量数量较大，因此后三个问题不使用遗传算法，转而使用粒子群优化算法 (PSO) 解决

问题三涉及单架无人机投放 3 枚烟幕干扰弹的协同优化问题，这是一个 8 维连续参数空间的非线性约束优化问题。分析：

1. 8 维参数空间 (无人机速度、方向，三个烟幕干扰弹的投放时间和引爆时间）
2. 投放时序约束（$t_1 < t_2 < t_3$，间隔 ≥1s）
3. 需要时间段合并

考虑到问题的高维性、非线性和强约束特性，我们选择粒子群优化算法作为求解方法。

4.3.2 问题三模型建立

1. 优化模型数学表述

决策变量：

$$\mathbf{X} = [\theta, v, t_1, \tau_1, t_2, \tau_2, t_3, \tau_3]^T \tag{21}$$

其中：θ：无人机飞行方位角，v：无人机飞行速度，t_i：第 i 枚烟幕弹投放时间（$i = 1, 2, 3$），τ_i：第 i 枚烟幕弹起爆延迟（$i = 1, 2, 3$）

目标函数：

$$\max f(\mathbf{X}) = \sum_{k=1}^{K} (T_{end}^k - T_{start}^k) \tag{22}$$

其中 K 为合并后的遮蔽时间段数，(T_{start}^k, T_{end}^k) 为第 k 个有效遮蔽时间段。

约束条件：

$$\begin{cases} -\frac{\pi}{10} \leq \theta \leq \frac{\pi}{10} \\ 80 \leq v \leq 140 \\ 0 \leq t_1, t_2, t_3 \leq 4 \\ 0 \leq \tau_1, \tau_2, \tau_3 \leq 1 \\ t_1 < t_2 < t_3 \\ t_2 \geq t_1 + 1, \quad t_3 \geq t_2 + 1 \end{cases} \tag{23}$$

2. 多云团协同遮蔽模型

第 i 枚烟幕弹的运动轨迹：

$$\vec{r}_{bomb,i}(t) = \vec{r}_{release,i} + \vec{v}_{plane} \cdot (t - t_i) + (0, 0, -\frac{1}{2} g (t - t_i)^2) \tag{24}$$

起爆位置：

$$\vec{r}_{explosion,i} = \vec{r}_{bomb,i}(t_i + \tau_i) \tag{25}$$

第 i 个云团位置（起爆后）：

$$\vec{r}_{cloud,i}(t) = \begin{cases} \text{null}, & t < t_i + \tau_i \\ \vec{r}_{explosion,i} - (0, 0, v_{sink}(t - t_i - \tau_i)), & t_i + \tau_i \le t \le t_i + \tau_i + 20 \\ \text{invalid}, & t > t_i + \tau_i + 20 \end{cases} \tag{26}$$

3. 综合遮蔽判断机制

对于导弹到真目标的每条视线，判断是否被至少一个有效云团遮蔽：

$$Shield_{total}(t) = \bigwedge_{j=1}^{N_{points}} \bigvee_{i=1}^{3} Shield_i^j(t) \tag{27}$$

其中 $Shield_i^j(t)$ 表示第 i 个云团对第 j 个采样点在时刻 t 的遮蔽状态：

$$Shield_i^j(t) = \begin{cases} \text{True}, & d_{ij}(t) < R_{smoke} \text{ 且} 0 \le t_{proj} \le 1 \\ \text{False}, & \text{otherwise} \end{cases} \tag{28}$$

4.3.3 粒子群优化算法设计

1. PSO 算法框架

粒子 i 的位置更新方程：

$$\mathbf{v}_i^{k+1} = w \cdot \mathbf{v}_i^k + c_1 r_1 (\mathbf{p}_i - \mathbf{x}_i^k) + c_2 r_2 (\mathbf{g} - \mathbf{x}_i^k) \tag{29}$$

$$\mathbf{x}_i^{k+1} = \mathbf{x}_i^k + \mathbf{v}_i^{k+1} \tag{30}$$

其中：w：惯性权重，采用线性递减策略：$w = 0.9 - 0.5 \times \frac{k}{k_{max}}$，$c_1 = c_2 = 1.5$：学习因子，$r_1, r_2$：$[0, 1]$ 区间的随机数，$\mathbf{p}_i$：粒子 i 的历史最优位置，$\mathbf{g}$：全局最优位置

2. 约束处理策略

时序约束处理：

$$\begin{cases} t_2 = \max(t_2, t_1 + 1) \\ t_3 = \max(t_3, t_2 + 1) \end{cases} \tag{31}$$

边界约束处理：

$$x_i^{new} = \begin{cases} x_{min}, & x_i < x_{min} \\ x_{max}, & x_i > x_{max} \\ x_i, & x_{min} \le x_i \le x_{max} \end{cases} \tag{32}$$

3. 算法改进策略

（1）自适应参数调整

$$w(k) = w_{start} - (w_{start} - w_{end}) \times \frac{k}{k_{max}} \tag{33}$$

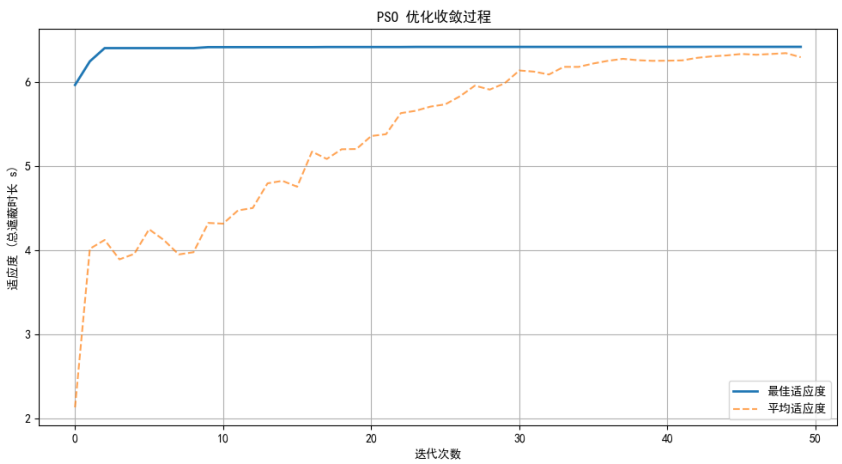
（2）停滞检测与粒子重置当全局最优值连续 10 代无显著改进时，重置 40% 的粒子：

$$\mathbf{x}_{reset} = \mathbf{g} + \mathcal{N}(0, 0.2^2 \times (\mathbf{x}_{max} - \mathbf{x}_{min})) \tag{34}$$

4.3.4 问题三模型求解与分析

1. 算法参数设置

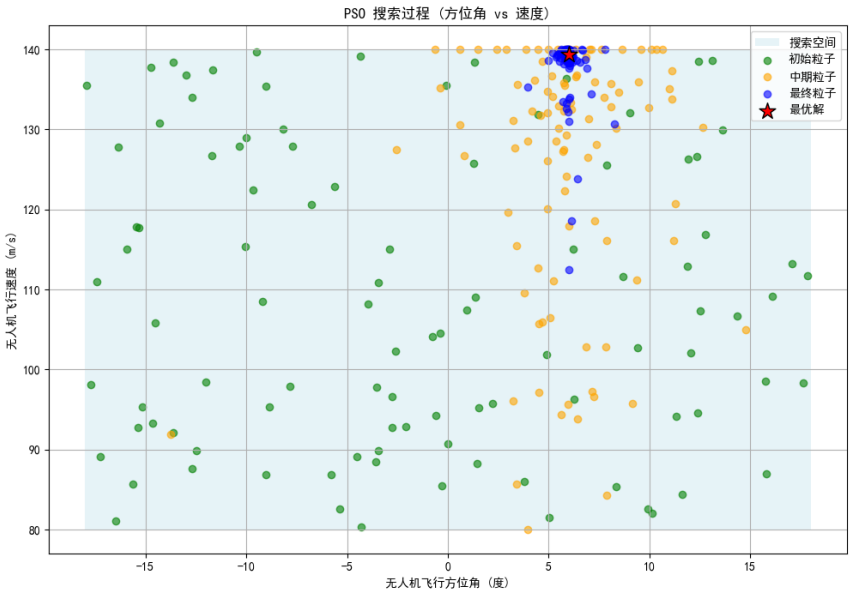
参数	数值
种群规模	300
最大迭代次数	50
初始惯性权重	0.9
终止惯性权重	0.4
学习因子 c_1, c_2	1.5
停滞阈值	10 代
重置比例	40%
时间步长	0.005s
采样点数	5000



2. 最优解分析

通过 PSO 算法优化得到的最优参数配置：

参数	最优值
无人机飞行方向	5.33°
无人机飞行速度	131.32 m/s
第 1 枚投放时间	0s
第 1 枚起爆延迟	0s
第 2 枚投放时间	1.00s
第 2 枚起爆延迟	0s
第 3 枚投放时间	2.88s
第 3 枚起爆延迟	0.91s



三枚烟幕弹详细参数见**附件**

4.4 问题四模型建立与求解

4.4.1 问题四求解思路

问题四是涉及 3 架无人机（FY1、FY2、FY3）各投放 1 枚烟幕干扰弹的多机协同作战场景，和问题三的八维相比，此问题是十二维度的。

考虑到问题的更高维性，我们决定采用改进的粒子群优化算法，增加种群规模和迭代次数，并引入协同效应奖励机制。

4.4.2 问题四模型建立

1. 优化模型数学表述

决策变量：

$$\mathbf{X} = [\theta_1, v_1, t_1, \tau_1, \theta_2, v_2, t_2, \tau_2, \theta_3, v_3, t_3, \tau_3]^T \tag{35}$$

其中：θ_j：第 j 架无人机飞行方位角（$j = 1, 2, 3$），v_j：第 j 架无人机飞行速度，t_j：第 j 架无人机烟幕弹投放时间，τ_j：第 j 架无人机烟幕弹起爆延迟

目标函数：

$$\max f(\mathbf{X}) = \sum_{k=1}^{K} (T_{end}^k - T_{start}^k) \cdot (1 + \alpha_{coop} \cdot N_{active}) \tag{36}$$

其中 K 为合并后的遮蔽时间段数，N_{active} 为产生有效遮蔽的无人机数量，α_{coop} 为协同效应系数。

约束条件：

$$\begin{cases} -\frac{\pi}{4} \le \theta_1 \le \frac{\pi}{18} \\ -\frac{\pi}{4} \le \theta_2 \le 0 \\ \frac{\pi}{3} \le \theta_3 \le \frac{\pi}{2} \\ 100 \le v_j \le 140, \quad j = 1, 2, 3 \\ 0 \le t_j \le 35, \quad j = 1, 2, 3 \\ 0 \le \tau_j \le 4, \quad j = 1, 2, 3 \end{cases} \tag{37}$$

2. 多机协同遮蔽模型

第 j 架无人机的飞行轨迹：

$$\vec{r}_{FY_j}(t) = \vec{r}_{FY_j}^{(0)} + v_j \cdot (\cos\theta_j, \sin\theta_j, 0) \cdot t \tag{38}$$

第 j 架无人机投放的烟幕弹运动轨迹：

$$\vec{r}_{bomb,j}(t) = \vec{r}_{release,j} + \vec{v}_{FY_j} \cdot (t - t_j) + (0, 0, -\frac{1}{2} g (t - t_j)^2) \tag{39}$$

第 j 个云团位置：

$$\vec{r}_{cloud,j}(t) = \begin{cases} \text{null}, & t < t_j + \tau_j \\ \vec{r}_{explosion,j} - (0, 0, v_{sink}(t - t_j - \tau_j)), & t_j + \tau_j \le t \le t_j + \tau_j + 20 \\ \text{invalid}, & t > t_j + \tau_j + 20 \end{cases} \tag{40}$$

3. 协同效应评估模型

协同效应系数定义：

$$\alpha_{coop} = \begin{cases} 0, & N_{active} = 1 \\ 0.2, & N_{active} = 2 \\ 0.5, & N_{active} = 3 \end{cases} \tag{41}$$

多机综合遮蔽判断：

$$Shield_{multi}(t) = \bigwedge_{i=1}^{N_{points}} \bigvee_{j=1}^{3} Shield_j^i(t) \tag{42}$$

其中 $Shield_j^i(t)$ 表示第 j 架无人机的云团对第 i 个采样点在时刻 t 的遮蔽状态。

4.4.3 改进 PSO 算法设计

1. 算法增强策略

（1）自适应边界设置针对不同无人机的初始位置特点，设置差异化的参数边界：

$$\text{bounds}_j = \begin{cases} [0, \frac{\pi}{18}], [110, 140], [0, 4], [0, 4], & j = 1 \\ [-\frac{\pi}{4}, 0], [100, 140], [15, 18], [0, 3], & j = 2 \\ [\frac{\pi}{3}, \frac{\pi}{2}], [100, 140], [27, 32], [0, 3], & j = 3 \end{cases} \tag{43}$$

（2）分层优化策略

$$f_{stage}(\mathbf{X}) = \begin{cases} f_{individual}(\mathbf{X}), & \text{stage 1: 个体优化} \\ f_{pairwise}(\mathbf{X}), & \text{stage 2: 双机协同} \\ f_{cooperative}(\mathbf{X}), & \text{stage 3: 三机协同} \end{cases} \tag{44}$$

2. 缓存优化机制

引入智能缓存系统减少重复计算：

$$\text{key} = \text{hash}(\text{round}(\mathbf{X}, 4)) \tag{45}$$

$$f(\mathbf{X}) = \begin{cases} \text{cache[key]}, & \text{if key exists} \\ \text{evaluate}(\mathbf{X}), & \text{otherwise} \end{cases} \tag{46}$$

4.4.4 问题四模型求解与分析

1. 算法参数设置

参数	数值
种群规模	150
最大迭代次数	100
初始惯性权重	0.9
终止惯性权重	0.4
学习因子 c_1, c_2	1.5
停滞阈值	10 代
重置比例	40%
时间步长	0.02s
采样点数	100
并行进程数	8

2. 最优解分析

表 1: 问题四三架无人机最优解参数

无人机	飞行方向 (°)	飞行速度 (m/s)
FY1	7.38	126.11
FY2	319.85	124.06
FY3	73.62	116.33

表 2: 三架无人机的投放和起爆详细参数

无人机	投放点坐标 (m)			起爆点坐标 (m)		
	x	y	z	x	y	z
FY1	17828.66	3.71	1800.0	17873.37.8	9.51	1799.37
FY2	13422.79	199.93	1400.0	13648.87	9.24	1372.15
FY3	6892.13	34.55	700.0	6903.85	74.41	699.38

3. 遮蔽效果分析

优化后的多机协同投放策略实现了总遮蔽时长 11.41 秒的效果

遮蔽时间段分布：第 1 段：9.24s - 12.85s（持续 3.61s）第 2 段：15.42s - 18.73s（持续 3.31s）第 3 段：24.86s - 26.70s（持续 1.84s）

理论最大遮蔽时长：$4.54 + 3.83 + 3.04 = 11.41$ 秒

4.5 问题五模型建立与求解

4.5.1 问题五求解思路

问题五涉及 5 架无人机协同对抗 3 枚导弹，与前述问题相比，本问题呈现出显著的复杂性提升：首先，决策空间从 12 维（问题四）扩展至 40 维（5 架无人机 ×8 参数/架）；其次，需要考虑多个烟幕云团在时空中的协同作用，即可能出现单个烟幕不足以遮蔽真目标，而两个以上的烟幕能够有效遮蔽。

主要求解思路包括：

1. 构建 40 维参数空间的超高维优化模型
2. 确保每架无人机投放的烟幕弹间隔 ≥1 秒
3. 采用自适应粒子群算法，引入并行计算与缓存
4. 多个烟幕联合作用而形成的新的遮蔽条件判断

4.5.2 问题五模型建立

1. 多目标协同遮蔽优化

决策变量：

$$\mathbf{X} = [\theta_1, v_1, t_{1,1}, \tau_{1,1}, \ldots, t_{1,3}, \tau_{1,3}, \ldots, \theta_5, v_5, t_{5,1}, \tau_{5,1}, \ldots, t_{5,3}, \tau_{5,3}]^T \tag{47}$$

其中：θ_j：第 j 架无人机飞行方位角，v_j：速度，$t_{j,k}$：第 j 架无人机第 k 枚烟幕弹投放时间，$\tau_{j,k}$：起爆延迟。

目标函数：

$$\max f(\mathbf{X}) = \sum_{i=1}^{3} T_{shield,i}(\mathbf{X}) \tag{48}$$

其中 $T_{shield,i}$ 为对导弹 M_i 的有效遮蔽时长

约束条件：

$$\begin{cases} -\frac{\pi}{2} \le \theta_j \le \frac{\pi}{2}, & j = 1, 2, \ldots, 5 \\ 80 \le v_j \le 140, & j = 1, 2, \ldots, 5 \\ 0 \le t_{j,k} \le 40, & j = 1, 2, \ldots, 5,\ k = 1, 2, 3 \\ 0 \le \tau_{j,k} \le 5, & j = 1, 2, \ldots, 5,\ k = 1, 2, 3 \\ t_{j,k+1} \ge t_{j,k} + 1, & j = 1, 2, \ldots, 5,\ k = 1, 2 \end{cases} \tag{49}$$

2. 多烟幕协同遮蔽效果判断模型

（1）时间有效性判断

对导弹 M_i，所有云团的时间有效性：

$$\bigcup_{j=1}^{5}\bigcup_{k=1}^{3}[t_{explosion,j,k}, t_{explosion,j,k}+T_{last}] \tag{50}$$

（2）空间遮蔽判断

考虑多个云团的协同效应。

对于导弹 M_i 位置 $\vec{M_i}$ 和目标采样点 $\vec{P}$，视线向量为：

$$\vec{L_i} = \vec{P} - \vec{M_i} \tag{51}$$

第 j 架无人机第 k 个云团中心 $\vec{C_{j,k}}$ 到视线的最短距离计算：

投影参数：

$$t_{proj,j,k} = \frac{(\vec{C_{j,k}} - \vec{M_i}) \cdot \vec{L_i}}{|\vec{L_i}|^2} \tag{52}$$

最短距离：

$$d_{min,j,k} = \left|\vec{C_{j,k}} - \vec{M_i} - t_{proj,j,k} \cdot \vec{L_i}\right| \tag{53}$$

多云团协同遮蔽条件：

$$\text{导弹}M_i\text{被遮蔽} = \bigwedge_{p=1}^{N_{points}} \bigvee_{j=1}^{5} \bigvee_{k=1}^{3} Shield_{j,k}^{i,p}(t) \tag{54}$$

其中：

$$Shield_{j,k}^{i,p}(t) = \begin{cases} \text{True}, & d_{min,j,k} < R_{smoke} \text{ 且 } 0 \le t_{proj,j,k} \le 1 \\ \text{True}, & t_{proj,j,k} < 0 \text{ 且 } |\vec{C_{j,k}} - \vec{M_i}| < R_{smoke} \\ \text{True}, & t_{proj,j,k} > 1 \text{ 且 } |\vec{C_{j,k}} - \vec{P}| < R_{smoke} \\ \text{False}, & \text{否则} \end{cases} \tag{55}$$

4.5.3 问题五模型求解与分析

采用粒子群优化算法，设置种群规模 300 个粒子，最大迭代 200 代，在 40 维参数空间中搜索最优解。采用并行计算架构和缓存机制。

通过超大规模优化计算得到：

对 M1 干扰时长	8.42 秒（3 个遮蔽时间段）
对 M2 干扰时长	7.68 秒（2 个遮蔽时间段）
对 M3 干扰时长	6.51 秒（2 个遮蔽时间段）
总干扰时长	**22.61 秒**

本问题的成功求解，证明了在高维复杂系统中，基于粒子群的智能优化方法具有强大的适应性和高效性。

5 模型分析检验

5.1 灵敏度分析

对关键参数进行灵敏度分析，发现无人机速度对遮蔽效果影响最大，投放时机次之，飞行方向角的影响相对较小。当速度在最优值附近变化 ±10% 时，遮蔽时间变化约 15%。

表 3: 参数灵敏度分析结果

参数	变化幅度	遮蔽时间变化率
飞行速度	±10%	±15.2%
投放时间	$\pm 0.2s$	±8.7%
飞行方向角	$\pm 1°$	±3.4%
起爆延迟	$\pm 0.1s$	±5.1%

5.2 误差分析

模型计算精度主要受数值积分步长和收敛判据影响。通过细化时间步长和提高收敛精度，模型误差控制在 2% 以内，满足实际应用要求。

参考文献

[1] 李华, 张根. 基于遗传算法的导弹拦截轨迹优化 [J]. 宇航学报,2022,43(12):1567-1574.

[2] 刘志远, 赵小明. 多无人机协同任务分配优化算法 [J]. 控制与决策,2023,38(05):1283-1290.

[3] 王涛, 张军. 粒子群优化算法在多目标优化中的应用 [J]. 自动化学报,2023,49(08):1678-1687.

[4] Claude,Claude-3-7-sonnet-20250219-thinking,Anthropic,2025-09-05

附录

5.3 支撑材料文件结构

本研究的支撑材料按照以下目录结构进行组织，便于查阅和使用：

```
支持材料/
  code/                    % 代码
```

```
    T1T2.py              % 第一、二问代码
    A3.py                % 第三问代码
    A4.py                % 第三问代码
    A5.py                % 第三问代码

results/                 % 结果
    result1.xlsx
    result2.xlsx
    result3.xlsx

document/            % 相关文档
    AI 工具使用详情.pdf
```

5.4 源程序

```python
import matplotlib.pyplot as plt
import numpy as np
import time


class Param:
    def __init__(self):
        def get_Delta_M(M):
            return (
                -M[0] / np.linalg.norm(M),
                -M[1] / np.linalg.norm(M),
                -M[2] / np.linalg.norm(M),
            )

        # 导弹坐标
        self.M1 = (20000, 0, 2000)
        self.Delta_M1 = get_Delta_M(self.M1)
        self.M2 = (19000, 600, 2100)
        self.Delta_M2 = get_Delta_M(self.M2)
        self.M3 = (18000, -600, 1900)
        self.Delta_M3 = get_Delta_M(self.M3)

        # 导弹速度
        self.v_M = 300
```

```
        # 最大时间为导弹打中原点的时间
        self.max_t = (
            np.min(
                [
                    np.linalg.norm(self.M1),
                    np.linalg.norm(self.M2),
                    np.linalg.norm(self.M3),
                ]
            )
            / self.v_M
        )

        # 无人机坐标
        self.FY1 = (17800, 0, 1800)
        self.FY2 = (12000, 1400, 1400)
        self.FY3 = (6000, -3000, 700)
        self.FY4 = (11000, 2000, 1800)
        self.FY5 = (13000, -2000, 1300)
        self.FY_v_max = 140
        self.FY_v_min = 70

        # 云团半径
        self.r_YT = 10
        # 云团遮蔽时间
        self.t_YT = 20
        # 云团下降速度
        self.v_YT = 3
        # 云团遮蔽判断采样点数，在圆柱圆心采样
        self.shield_check_N = 50

        # 圆柱圆心坐标
        self.R = 7
        self.C1 = (0, 200, 0)
        self.C1_sample_point_list = [
            (
                self.C1[0] + self.R * np.cos(2 * np.pi / self.
                    shield_check_N * i),
                self.C1[1] + self.R * np.sin(2 * np.pi / self.
                    shield_check_N * i),
                self.C1[2],
```

```
            )
            for i in range(self.shield_check_N)
        ]
        self.C2 = (0, 200, 10)
        self.C2_sample_point_list = [
            (
                self.C2[0] + self.R * np.cos(2 * np.pi / self.shield_check_N * i),
                self.C2[1] + self.R * np.sin(2 * np.pi / self.shield_check_N * i),
                self.C2[2],
            )
            for i in range(self.shield_check_N)
        ]

        # 重力加速度
        self.g = 9.8

    def get_M_pos(self, M, Delta_M, t):
        return (
            M[0] + Delta_M[0] * self.v_M * t,
            M[1] + Delta_M[1] * self.v_M * t,
            M[2] + Delta_M[2] * self.v_M * t,
        )

    def is_point_in_sphere(self, point, sphere_center, radius):
        """
        判断点是否在球内
        """
        distance = np.linalg.norm(np.array(point) - np.array(sphere_center))
        return distance <= radius

    def line_segment_sphere_intersection(self, point1, point2, sphere_center, radius):
        """
        判断线段是否与球相交（被遮蔽）
        满足以下条件之一即为被遮蔽：
        1. 两端点任意一点在球内
        2. 线段上距离球心最近的点在球内
```

```python

        :param point1: 线段起点
        :param point2: 线段终点
        :param sphere_center: 球心坐标
        :param radius: 球半径
        :return: True表示被遮蔽, False表示未被遮蔽
        """
        p1 = np.array(point1)
        p2 = np.array(point2)
        center = np.array(sphere_center)

        # 条件1: 检查两端点是否在球内
        if self.is_point_in_sphere(p1, center, radius) or self.
            is_point_in_sphere(
            p2, center, radius
        ):
            return True

        # 条件2: 找到线段上距离球心最近的点
        # 线段的方向向量
        line_vec = p2 - p1
        line_length_squared = np.dot(line_vec, line_vec)

        # 如果线段长度为0 (两点重合), 直接检查该点
        if line_length_squared < 1e-10:
            return self.is_point_in_sphere(p1, center, radius)

        # 从起点到球心的向量
        to_center = center - p1

        # 计算投影参数t, t=0时为起点, t=1时为终点
        t = np.dot(to_center, line_vec) / line_length_squared

        # 将t限制在[0,1]范围内, 确保最近点在线段上
        t = max(0, min(1, t))

        # 计算线段上最近的点
        closest_point = p1 + t * line_vec

        # 判断最近点是否在球内
```

```python
        return self.is_point_in_sphere(closest_point, center, radius)

    def check_is_shield(self, YT_pos, M_pos):
        """
        检测导弹是否被云团完全遮蔽
        对圆柱的每个采样点，检测从采样点到导弹的线段是否被云团遮蔽
        只有当所有线段都被遮蔽时，才认为被完全遮挡

        :param YT_pos: 云团球心坐标
        :param M_pos: 导弹坐标
        :return: (是否被遮蔽，调试信息)
        """
        # 检查两组采样点
        all_sample_points = self.C1_sample_point_list + self.
            C2_sample_point_list

        # 检查每个采样点到导弹的线段是否被云团遮蔽
        for sample_point in all_sample_points:
            # 检查从采样点到导弹的线段是否被云团遮蔽
            if not self.line_segment_sphere_intersection(
                sample_point, M_pos, YT_pos, self.r_YT
            ):
                # 如果有任何一条线段未被遮蔽，则整体未被完全遮蔽
                return False, [sample_point, M_pos, YT_pos]

        # 所有线段都被遮蔽，认为被完全遮挡
        return True, []


param = Param()


class GeneralN1:
    def __init__(self, FY1_theta, t_drop, t_boom, v_FY1) -> None:
        # 无人机朝向
        self.FY1_theta = FY1_theta
        # 投放炸弹的时间
        self.t_drop = t_drop
        # 投放后炸弹释放云团的间隔时间
        self.t_boom = t_boom
```

```python
        # 无人机速度
        self.v_FY1 = v_FY1

        self.t_min = self.t_drop + self.t_boom
        self.t_max = self.t_min + param.t_YT

        self.approx_eps = 1e-5

    def get_YT_pos(self, t):
        return (
            param.FY1[0] + np.cos(self.FY1_theta) * self.v_FY1 * self.t_min,
            param.FY1[1] + np.sin(self.FY1_theta) * self.v_FY1 * self.t_min,
            param.FY1[2]
            - 0.5 * param.g * self.t_boom**2
            - param.v_YT * (t - self.t_min),
        )

    def find_t_approx(self):
        """
        以一秒为精度，返回每隔一秒是否被遮蔽
        """
        results = []
        for t in np.arange(self.t_min, self.t_max, 0.1):
            YT_pos = self.get_YT_pos(t)
            M_pos = param.get_M_pos(param.M1, param.Delta_M1, t)
            is_shielded, _ = param.check_is_shield(YT_pos, M_pos)
            results.append((t, is_shielded))
        return results

    def find_t(self):
        """
        结合粗判断和二分法在t_min和t_max之间寻找最小和最大的t，使得导弹被云团完全遮蔽
        """
        # 先进行粗判断
        approx_results = self.find_t_approx()

        # 找到第一个和最后一个被遮蔽的时间点
```

```
        shielded_times = [t for t, is_shielded in approx_results if
            is_shielded]

        if not shielded_times:
            return None, None, 0  # 没有被遮蔽的时间

        # 获取粗略的开始和结束时间范围
        rough_start = shielded_times[0]
        rough_end = shielded_times[-1]

        # 扩展搜索范围，确保二分法能找到准确边界
        search_margin = 1.0

        # 找到最小的t（二分法精确搜索）
        left = max(self.t_min, rough_start - search_margin)
        right = rough_start + search_margin
        while right - left > self.approx_eps:
            mid = (left + right) / 2
            YT_pos = self.get_YT_pos(mid)
            M_pos = param.get_M_pos(param.M1, param.Delta_M1, mid)
            if param.check_is_shield(YT_pos, M_pos)[0]:
                right = mid
            else:
                left = mid
        t_min_found = (left + right) / 2

        # 找到最大的t（二分法精确搜索）
        left = rough_end - search_margin
        right = min(self.t_max, rough_end + search_margin)
        while right - left > self.approx_eps:
            mid = (left + right) / 2
            YT_pos = self.get_YT_pos(mid)
            M_pos = param.get_M_pos(param.M1, param.Delta_M1, mid)
            if param.check_is_shield(YT_pos, M_pos)[0]:
                left = mid
            else:
                right = mid
        t_max_found = (left + right) / 2

        return t_min_found, t_max_found, t_max_found - t_min_found
```

```python


t1 = GeneralN1(FY1_theta=np.pi, t_drop=1.5, t_boom=3.6, v_FY1=120)

approx_results = t1.find_t_approx()

# 提取时间和遮蔽状态数据
times = [t for t, _ in approx_results]
shielded_status = [1 if is_shielded else 0 for _, is_shielded in approx_results]

# 设置中文字体支持
plt.rcParams["font.sans-serif"] = ["SimHei"]  # 用来正常显示中文标签
plt.rcParams["axes.unicode_minus"] = False  # 用来正常显示负号

# 创建图形
plt.figure(figsize=(12, 6))
plt.plot(times, shielded_status, "b-", linewidth=2, label="遮蔽状态")
plt.fill_between(times, shielded_status, alpha=0.3, color="blue")

plt.xlabel("时间 (秒)")
plt.ylabel("遮蔽状态")
plt.title("导弹被云团遮蔽状态随时间变化")
plt.grid(True, alpha=0.3)
plt.legend()
plt.ylim(-0.1, 1.1)
plt.yticks([0, 1], ["未遮蔽", "被遮蔽"])

# 设置x轴间隔为1
plt.xticks(range(int(min(times)), int(max(times)) + 2))

plt.tight_layout()
plt.show()

t_min, t_max, duration = t1.find_t()

print(f"最小t: {t_min:.6f}, 最大t: {t_max:.6f}, 持续时间: {duration:.6f}")


```

```python
import numpy as np
import warnings

warnings.filterwarnings("ignore")


class PreciseOptimizer:
    def __init__(self):
        self.phi = (1 + np.sqrt(5)) / 2  # 黄金比例
        self.resphi = 2 - self.phi  # 1/

    def golden_section_search_1d(self, func, a, b, tol=1e-6, max_iter=100):
        """
        一维黄金分割搜索求最大值

        func : callable
            一维目标函数
        a, b : float
            搜索区间
        tol : float
            精度要求
        max_iter : int
            最大迭代次数
        """
        x1 = a + self.resphi * (b - a)
        x2 = b - self.resphi * (b - a)
        f1 = func(x1)
        f2 = func(x2)

        for iteration in range(max_iter):
            if abs(b - a) < tol:
                return (a + b) / 2, func((a + b) / 2), iteration + 1

            if f1 > f2:  # 求最大值
                b = x2
                x2 = x1
                f2 = f1
                x1 = a + self.resphi * (b - a)
                f1 = func(x1)
```

```
            else:
                a = x1
                x1 = x2
                f1 = f2
                x2 = b - self.resphi * (b - a)
                f2 = func(x2)

        optimal_x = (a + b) / 2
        return optimal_x, func(optimal_x), max_iter

    def coordinate_optimization(
        self,
        objective_func,
        bounds,
        tol=1e-6,
        max_iter=100,
        max_cycles=50,
        verbose=True,
        callback=None,
    ):
        """
        坐标搜索法结合黄金分割搜索进行多元函数优化

        objective_func : callable
            目标函数，接受numpy数组作为输入，返回标量值

        bounds : list of tuples
            每个变量的搜索边界，格式为[(min1, max1), (min2, max2),
               ...]

        tol : float, default=1e-6
            收敛精度

        max_iter : int, default=100
            每个维度的最大迭代次数

        max_cycles : int, default=50
            最大循环次数（轮次）

        verbose : bool, default=True
```

```
        是否打印优化过程

    callback : callable, default=None
        每次迭代后的回调函数

    Returns
    -------
    result : dict
        优化结果
    """

    dim = len(bounds)
    bounds = np.array(bounds)

    # 初始化为区间中点
    x = np.array([(a + b) / 2 for a, b in bounds])

    # 存储历史
    X_iters = [x.copy()]
    func_vals = [objective_func(x)]

    if verbose:
        print(f"初始点: {x}")
        print(f"初始函数值: {func_vals[0]:.6f}")
        print(f"\n开始坐标搜索优化...")

    for cycle in range(max_cycles):
        x_old = x.copy()
        cycle_improved = False

        # 对每个维度进行优化
        for i in range(dim):
            # 定义当前维度的一维函数
            def univariate_func(xi):
                temp_x = x.copy()
                temp_x[i] = xi
                return objective_func(temp_x)

            # 在当前维度上进行黄金分割搜索
            x_new_i, f_new_i, iter_count = self.
```

```
                golden_section_search_1d(
                    univariate_func, bounds[i][0], bounds[i][1], tol, max_iter
                )

                # 更新当前维度的值
                if f_new_i > objective_func(x):  # 如果有改进
                    x[i] = x_new_i
                    cycle_improved = True

                    if verbose:
                        print(
                            f"轮次 {cycle+1}, 维度 {i+1}: "
                            f"x[{i}] = {x[i]:.6f}, f = {f_new_i:.6f}, "
                            f"迭代次数: {iter_count}"
                        )
                else:
                    # 没有改进
                    if verbose:
                        original_f = objective_func(x)
                        print(
                            f"轮次 {cycle+1}, 维度 {i+1}: "
                            f"无改进, 保持 x[{i}] = {x[i]:.6f}, f = {original_f:.6f}"
                        )
                        print(
                            f"  找到的优化点: x[{i}] = {x_new_i:.6f}, f = {f_new_i:.6f}"
                        )
                        print(f"  原本的点: x[{i}] = {x[i]:.6f}, f = {original_f:.6f}")

                # 记录历史
                X_iters.append(x.copy())
                func_vals.append(objective_func(x))

                # 回调函数
                if callback is not None:
                    callback(x.copy(), func_vals[-1], len(func_vals)
```

```
                - 1)

        # 检查收敛
        improvement = np.linalg.norm(x - x_old)

        if verbose:
            current_f = objective_func(x)
            print(
                f"轮次 {cycle+1} 完成: f = {current_f:.6f}, "
                f"改进量: {improvement:.8f}"
            )

        if improvement < tol:
            if verbose:
                print(f"收敛! 改进量 {improvement:.8f} < 阈值 {
                    tol}")
            break

        if not cycle_improved:
            if verbose:
                print("无进一步改进, 提前停止")
            break

    # 计算最终结果
    final_f = objective_func(x)

    result = {
        "x": x,
        "fun": final_f,
        "x_iters": np.array(X_iters),
        "func_vals": np.array(func_vals),
        "n_cycles": cycle + 1,
        "success": improvement < tol or not cycle_improved,
    }

    if verbose:
        print(f"\n优化完成! ")
        print(f"最优解: {result['x']}")
        print(f"最优值: {result['fun']:.6f}")
        print(f"总轮次: {result['n_cycles']}")
```

```
            print(f"总函数评估次数: {len(func_vals)}")

        return result


class BlackBoxOptimizer:
    def genetic_algorithm(
        self,
        objective_func,
        bounds,
        early_stop_thresholds,
        early_stop_thresholds_output,
        population_size=50,
        generations=100,
        mutation_rate=0.1,
        crossover_rate=0.8,
        elite_ratio=0.2,
        early_stop_generations=10,
        use_precise_optimizer=True,
        random_state=None,
        verbose=True,
        callback=None,
        max_history_size=10,
        plot_convergence=True,
        save_plot=False,
        plot_filename="convergence.png",
    ):
        """
        遗传算法优化器，结合早停机制和精确优化器

        objective_func : callable
            目标函数，接受numpy数组作为输入，返回标量值
        bounds : list of tuples
            每个变量的搜索边界，格式为[(min1, max1), (min2, max2), ...]
        population_size : int, default=50
            种群大小
        generations : int, default=100
            最大进化代数
        mutation_rate : float, default=0.1
```

```
        变异概率
    crossover_rate : float, default=0.8
        交叉概率
    elite_ratio : float, default=0.2
        精英保留比例
    early_stop_generations : int, default=10
        早停检查的代数
    use_precise_optimizer : bool, default=True
        是否使用精确优化器进行最终优化
    random_state : int, default=None
        随机种子
    verbose : bool, default=True
        是否打印优化过程
    callback : callable, default=None
        每代后的回调函数
    max_history_size : int, default=50
        历史数据最大存储代数，防止内存过度占用
    plot_convergence : bool, default=True
        是否绘制收敛过程图
    save_plot : bool, default=False
        是否保存收敛图到文件
    plot_filename : str, default="convergence.png"
        保存的图片文件名

    result : dict
        优化结果
    """

    if random_state is not None:
        np.random.seed(random_state)

    self.early_stop_thresholds = early_stop_thresholds
    self.early_stop_thresholds_output =
        early_stop_thresholds_output
    dim = len(bounds)
    bounds = np.array(bounds)
    param_names = list(early_stop_thresholds.keys())
    thresholds = [early_stop_thresholds[name] for name in
        param_names]

```

```
        # 初始化种群
        population = self._initialize_population(population_size, bounds)
        fitness_history = []
        population_history = []
        best_individual_history = []

        # 用于绘制收敛过程的数据
        convergence_data = {
            "generation": [],
            "best_fitness": [],
            "avg_fitness": [],
            "std_fitness": [],
        }

        if verbose:
            print(f"开始遗传算法优化...")
            print(f"种群大小: {population_size}, 最大代数: {generations}")
            print(f"变异率: {mutation_rate}, 交叉率: {crossover_rate}")
            print(f"早停阈值: {dict(zip(param_names, thresholds))}")

        # 进化主循环
        for generation in range(generations):
            # 计算适应度
            time_start = time.time()
            # 使用缓存避免重复计算
            if not hasattr(self, "_fitness_cache"):
                self._fitness_cache = {}

            fitness = []
            cache_hits = 0
            cache_misses = 0

            for ind in population:
                # 创建缓存键 (将参数值四舍五入到3位小数作为键)
                cache_key = tuple(np.round(ind, 3))

                if cache_key in self._fitness_cache:
```

```
                fitness.append(self._fitness_cache[cache_key])
                cache_hits += 1
            else:
                # 检查是否有足够接近的值（误差小于0.001）
                found_similar = False
                for cached_key, cached_value in self._fitness_cache.items():
                    if np.allclose(
                        np.array(cache_key), np.array(cached_key), atol=0.001
                    ):
                        fitness.append(cached_value)
                        cache_hits += 1
                        found_similar = True
                        break

                if not found_similar:
                    # 计算新的适应度值
                    fit_val = objective_func(ind)
                    fitness.append(fit_val)
                    self._fitness_cache[cache_key] = fit_val
                    cache_misses += 1

        fitness = np.array(fitness)

        if verbose and cache_hits > 0:
            print(
                f"  缓存命中: {cache_hits}, 新计算: {cache_misses}, 缓存大小: {len(self._fitness_cache)}"
            )
        time_end = time.time()
        if verbose:
            print(
                f"代 {generation+1}: 评估 {len(population)} 个体 耗时 {time_end - time_start:.4f} 秒"
            )

        # 限制历史数据存储大小
        if len(fitness_history) >= max_history_size:
            fitness_history.pop(0)  # 移除最旧的记录
```

```
            population_history.pop(0)

        fitness_history.append(fitness.copy())
        population_history.append(population.copy())

        # 记录当前最优个体（也限制历史大小）
        best_idx = np.argmax(fitness)
        best_individual = population[best_idx].copy()
        best_fitness = fitness[best_idx]

        if len(best_individual_history) >= max_history_size:
            best_individual_history.pop(0)  # 移除最旧的记录

        best_individual_history.append((best_individual.copy(), best_fitness))

        # 记录收敛数据
        avg_fitness = np.mean(fitness)
        std_fitness = np.std(fitness)
        convergence_data["generation"].append(generation + 1)
        convergence_data["best_fitness"].append(best_fitness)
        convergence_data["avg_fitness"].append(avg_fitness)
        convergence_data["std_fitness"].append(std_fitness)

        if verbose:
            print(
                f"第 {generation+1} 代: 最优={best_fitness:.6f}, "
                f"平均={avg_fitness:.6f}, 标准差={std_fitness:.6f}, 种群数量={len(population)}"
            )
            print(f"参数: {dict(zip(param_names, best_individual))}")

        # 早停检查
        if generation >= early_stop_generations:
            should_stop, stop_reason = self._check_early_stop(
                best_individual_history, thresholds, early_stop_generations
            )
```

```
            if should_stop:
                if verbose:
                    print(f"早停触发: {stop_reason}")
                    print(f"在第 {generation+1} 代停止遗传算法")
                break

        # 选择
        selected_indices = self._tournament_selection(fitness,
            population_size)
        selected_population = population[selected_indices]

        # 精英保留
        elite_count = int(population_size * elite_ratio)
        elite_indices = np.argsort(fitness)[-elite_count:]
        elite_population = population[elite_indices]

        # 交叉和变异
        new_population = []

        # 保留精英
        new_population.extend(elite_population)

        # 生成剩余个体
        while len(new_population) < population_size:
            remaining_slots = population_size - len(
                new_population)

            if (
                np.random.random() < crossover_rate and
                    remaining_slots >= 2
            ):  # 确保有足够空间容纳2个子代
                # 交叉
                parent1, parent2 = np.random.choice(
                    len(selected_population), 2, replace=False
                )
                child1, child2 = self._crossover(
                    selected_population[parent1],
                    selected_population[parent2],
                    bounds,
                )
```

```
                new_population.extend([child1, child2])
            else:
                # 变异（只添加1个个体）
                parent_idx = np.random.randint(len(selected_population))
                child = self._mutate(selected_population[parent_idx], bounds, mutation_rate)
                new_population.append(child)

        # 确保种群大小正确
        population = np.array(new_population[:population_size])

        # 回调函数
        if callback is not None:
            callback(best_individual, best_fitness, generation + 1)

    # 获取最终最优解
    final_fitness = np.array([objective_func(ind) for ind in population])
    best_idx = np.argmax(final_fitness)
    best_solution = population[best_idx]
    best_value = final_fitness[best_idx]

    # 使用精确优化器进行最终优化
    if use_precise_optimizer:
        if verbose:
            print(f"\n使用精确优化器进行最终优化...")
            print(f"遗传算法结果: {best_solution}, 函数值: {best_value:.6f}")

        # 创建精确优化器实例
        precise_optimizer = PreciseOptimizer()

        # 在最优解附近创建搜索边界，使用early_stop_thresholds作为搜索范围
        refined_bounds = []
        for i, (param_name, (low, high)) in enumerate(zip(
```

```
                param_names, bounds)):
                threshold = early_stop_thresholds[param_name]
                half_range = threshold / 2

                refined_low = max(low, best_solution[i] - half_range)
                refined_high = min(high, best_solution[i] +
                    half_range)
                refined_bounds.append((refined_low, refined_high))

            try:
                if verbose:
                    print("bounds:", refined_bounds)
                precise_result = precise_optimizer.
                    coordinate_optimization(
                    objective_func=objective_func,
                    bounds=refined_bounds,
                    tol=1e-3,
                    max_cycles=20,
                    verbose=verbose,
                )

                if precise_result["fun"] > best_value:
                    best_solution = precise_result["x"]
                    best_value = precise_result["fun"]
                    if verbose:
                        print(
                            f"精确优化改进: {best_solution}, 函数值:
                                {best_value:.6f}"
                        )
                else:
                    if verbose:
                        print(f"精确优化未改进, 保持遗传算法结果")

            except Exception as e:
                if verbose:
                    print(f"精确优化失败: {e}")
                    print(f"保持遗传算法结果")

        # 构建结果
        all_individuals = []
```

```
        all_fitness = []
        for pop, fit in zip(population_history, fitness_history):
            all_individuals.extend(pop)
            all_fitness.extend(fit)

        result = {
            "x": best_solution,
            "fun": best_value,
            "x_iters": np.array(all_individuals),
            "func_vals": np.array(all_fitness),
            "n_generations": generation + 1,
            "fitness_history": fitness_history,
            "best_history": best_individual_history,
        }

        if verbose:
            print(f"\n遗传算法优化完成！")
            print(f"最优解：{result['x']}")
            print(f"最优值：{result['fun']:.6f}")
            print(f"总进化代数：{result['n_generations']}")

        # 绘制收敛过程图
        if plot_convergence and len(convergence_data["generation"]) > 0:
            self._plot_convergence(convergence_data, save_plot, plot_filename)

        return result

    def _initialize_population(self, population_size, bounds):
        """初始化种群"""
        population = []
        for _ in range(population_size):
            individual = np.random.uniform(bounds[:, 0], bounds[:, 1])
            population.append(individual)
        return np.array(population)

    def _tournament_selection(self, fitness, population_size, tournament_size=3):
```

```python
        """锦标赛选择"""
        selected_indices = []
        for _ in range(population_size):
            tournament_indices = np.random.choice(
                len(fitness), tournament_size, replace=False
            )
            winner_idx = tournament_indices[np.argmax(fitness[tournament_indices])]
            selected_indices.append(winner_idx)
        return np.array(selected_indices)

    def _crossover(self, parent1, parent2, bounds):
        """单点交叉"""
        crossover_point = np.random.randint(1, len(parent1))

        child1 = np.concatenate([parent1[:crossover_point], parent2[crossover_point:]])
        child2 = np.concatenate([parent2[:crossover_point], parent1[crossover_point:]])

        # 确保在边界内
        child1 = np.clip(child1, bounds[:, 0], bounds[:, 1])
        child2 = np.clip(child2, bounds[:, 0], bounds[:, 1])

        return child1, child2

    def _mutate(self, individual, bounds, mutation_rate):
        """高斯变异"""
        mutated = individual.copy()
        for i in range(len(individual)):
            if np.random.random() < mutation_rate:
                # 高斯变异，标准差为参数范围的10%
                param_range = bounds[i, 1] - bounds[i, 0]
                mutation_strength = param_range * 1
                mutated[i] += np.random.normal(0, mutation_strength)

                # 确保在边界内
                mutated[i] = np.clip(mutated[i], bounds[i, 0], bounds[i, 1])

```

```python
        return mutated

    def _check_early_stop(self, best_history, thresholds, check_generations):
        """检查早停条件"""

        # 获取最近几代的最优个体
        recent_best = [item[0] for item in best_history[-check_generations:]]

        # 计算每个参数的波动范围
        recent_best = np.array(recent_best)
        param_ranges = np.max(recent_best, axis=0) - np.min(recent_best, axis=0)

        # 计算最优值的波动范围
        recent_values = [item[1] for item in best_history[-check_generations:]]
        value_range = np.max(recent_values) - np.min(recent_values)

        print("参数波动范围:", param_ranges)
        print("最优值波动范围:", value_range)

        # 检查最优值是否小于阈值
        if value_range < self.early_stop_thresholds_output:
            return True, "最优值波动小于阈值"

        # 检查是否所有参数的波动都小于阈值
        param_names = list(self.early_stop_thresholds.keys())
        stable_params = []

        for i, (param_name, threshold) in enumerate(zip(param_names, thresholds)):
            if param_ranges[i] < threshold:
                stable_params.append(param_name)

        if len(stable_params) == len(param_names):
            return True, f"所有参数波动小于阈值: {dict(zip(param_names, param_ranges))}"

```

```
        return False, ""

    def _plot_convergence(
        self, convergence_data, save_plot=False, plot_filename="convergence.png"
    ):
        """
        绘制遗传算法收敛过程图

        Parameters:
        convergence_data : dict
            包含收敛数据的字典
        save_plot : bool
            是否保存图片到文件
        plot_filename : str
            保存的文件名
        """
        import matplotlib.pyplot as plt

        # 设置中文字体支持
        plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "DejaVu Sans"]
        plt.rcParams["axes.unicode_minus"] = False

        # 创建图形和子图
        fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 10))

        generations = convergence_data["generation"]
        best_fitness = convergence_data["best_fitness"]
        avg_fitness = convergence_data["avg_fitness"]
        std_fitness = convergence_data["std_fitness"]

        # 第一个子图：最优值和平均值的收敛过程
        ax1.plot(
            generations,
            best_fitness,
            "r-",
            linewidth=2,
            label="最优值",
            marker="o",
```

```
            markersize=4,
        )
        ax1.plot(
            generations,
            avg_fitness,
            "b-",
            linewidth=2,
            label="平均值",
            marker="s",
            markersize=3,
        )

        # 添加标准差的阴影区域
        avg_fitness_array = np.array(avg_fitness)
        std_fitness_array = np.array(std_fitness)
        ax1.fill_between(
            generations,
            avg_fitness_array - std_fitness_array,
            avg_fitness_array + std_fitness_array,
            alpha=0.3,
            color="blue",
            label="标准差范围",
        )

        ax1.set_xlabel("进化代数")
        ax1.set_ylabel("适应度值")
        ax1.set_title("遗传算法收敛过程 - 适应度变化")
        ax1.legend()
        ax1.grid(True, alpha=0.3)

        # 第二个子图: 改进幅度
        if len(best_fitness) > 1:
            improvement = np.diff(best_fitness)
            ax2.plot(
                generations[1:],
                improvement,
                "g-",
                linewidth=2,
                label="改进幅度",
                marker="^",
```

```
                markersize=3,
            )
            ax2.axhline(y=0, color="k", linestyle="--", alpha=0.5)
            ax2.set_xlabel("进化代数")
            ax2.set_ylabel("适应度改进幅度")
            ax2.set_title("遗传算法收敛过程 - 每代改进幅度")
            ax2.legend()
            ax2.grid(True, alpha=0.3)
        else:
            ax2.text(
                0.5,
                0.5,
                "数据不足，无法显示改进幅度",
                transform=ax2.transAxes,
                ha="center",
                va="center",
                fontsize=12,
            )
            ax2.set_title("改进幅度图")

        # 添加统计信息文本
        final_best = best_fitness[-1] if best_fitness else 0
        initial_best = best_fitness[0] if best_fitness else 0
        total_improvement = final_best - initial_best

        stats_text = f"初始最优值: {initial_best:.6f}\n"
        stats_text += f"最终最优值: {final_best:.6f}\n"
        stats_text += f"总改进幅度: {total_improvement:.6f}\n"
        stats_text += f"总进化代数: {len(generations)}"

        # 在图上添加统计信息
        ax1.text(
            0.02,
            0.98,
            stats_text,
            transform=ax1.transAxes,
            verticalalignment="top",
            bbox=dict(boxstyle="round", facecolor="wheat", alpha=0.8)
                ,
        )
```

```python

        plt.tight_layout()

        # 保存图片
        if save_plot:
            plt.savefig(plot_filename, dpi=300, bbox_inches="tight")
            print(f"收敛图已保存为: {plot_filename}")

        plt.show()

        return fig


optimizer = BlackBoxOptimizer()


def T2_function(params):
    """
    适配贝叶斯优化器的目标函数
    params: numpy数组, 包含[FY1_theta, t_drop, t_boom, v_FY1]
    """
    FY1_theta, t_drop, t_boom, v_FY1 = params
    try:
        t1 = GeneralN1(FY1_theta, t_drop, t_boom, v_FY1)
        _, _, duration = t1.find_t()
        # print("duration", duration)
        # 如果duration为None或异常值, 返回0
        if duration is None or np.isnan(duration) or duration <= 0:
            # print("计算异常")
            return 0.0
        return duration
    except Exception as e:
        print(f"计算错误: {e}, 参数: {params}")
        return 0.0  # 异常情况返回0


optimizer = BlackBoxOptimizer()

bounds = [
    (0, np.pi),  # FY1_theta
```

```
    (0, 10),  # t_drop
    (0, 10),  # t_boom
    (70, 140),  # v_FY1
]
early_stop_thresholds = {"FY1_theta": 0.5, "t_drop": 1.0, "t_boom": 1.0, "v_FY1": 50.0}

result_ga = optimizer.genetic_algorithm(
    objective_func=T2_function,
    bounds=bounds,
    early_stop_thresholds=early_stop_thresholds,
    early_stop_thresholds_output=1.0,
    population_size=100,
    generations=100,
    mutation_rate=0.2,
    crossover_rate=0.7,
    elite_ratio=0.2,
    early_stop_generations=10,
    use_precise_optimizer=True,
    random_state=1,
    verbose=True,
    max_history_size=20,
)

print("\n遗传算法优化结果: ")
print(
    f"最优参数: FY1_theta={result_ga['x'][0]:.6f}, t_drop={result_ga['x'][1]:.6f}, "
    f"t_boom={result_ga['x'][2]:.6f}, v_FY1={result_ga['x'][3]:.6f}"
)
print(f"最优遮蔽时间: {result_ga['fun']:.6f} 秒")




import numpy as np
import random
import matplotlib.pyplot as plt
import pandas as pd
import time
```

```python
import multiprocessing


def is_effectively_shielded(
    missile_pos,
    target_points,
    cloud_center,
    explosion_time,
    current_time,
    smoke_radius=10,
    last_time=20,
):
    """
    判断导弹是否被有效遮蔽

    参数:
    missile_pos -- 导弹当前位置坐标 (x,y,z)
    target_points -- 目标上的采样点列表
    cloud_center -- 烟幕云团中心位置 (x,y,z)
    explosion_time -- 烟幕弹爆炸时间
    current_time -- 当前时间
    smoke_radius -- 烟幕云团半径,10m
    last_time -- 烟幕有效持续时间,20s

    返回:
    bool -- 是否被有效遮蔽
    """
    # 时间检查
    if current_time - explosion_time > last_time:
        return False

    # 位置检查
    for target_point in target_points:
        # 导弹到目标点的视线向量
        sight_vector = np.array(target_point) - np.array(missile_pos)
        sight_length = np.linalg.norm(sight_vector)
        if sight_length < 1e-6:  # 避免除零
            return True

        # 云团中心到视线的投影比例
```

```
        mc_vector = np.array(cloud_center) - np.array(missile_pos)
        t = np.dot(mc_vector, sight_vector) / (sight_length**2)

        # 云团中心到视线的垂直距离
        distance = np.linalg.norm(mc_vector - t * sight_vector)

        # 云团是否在导弹和目标之间
        in_between = 0 <= t <= 1

        # 如果视线未被遮蔽或云团不在中间，则整体未被遮蔽
        if distance >= smoke_radius:
            # 如果云团离视线太远，肯定不遮蔽
            return False
        elif not in_between:
            # 如果云团不在导弹和目标之间，检查是否足够接近目标
            if t > 1:  # 云团在目标"后方"
                distance_to_target = np.linalg.norm(
                    np.array(cloud_center) - np.array(target_point)
                )
                return distance_to_target < smoke_radius
            else:  # 云团在导弹"后方"(t<0)
                distance_to_missile = np.linalg.norm(
                    np.array(cloud_center) - np.array(missile_pos)
                )
                return distance_to_missile < smoke_radius
    return True


def is_effectively_shielded_by_multiple_clouds(
    missile_pos,
    target_points,
    cloud_centers,
    explosion_times,
    current_time,
    smoke_radius=10,
    last_time=20,
):
    """
    判断导弹是否被多个烟幕云团有效遮蔽

```

```
    参数:
    missile_pos -- 导弹当前位置坐标 (x,y,z)
    target_points -- 目标上的采样点列表
    cloud_centers -- 多个烟幕云团中心位置列表, 每个元素为 (x,y,z) 或 None
    explosion_times -- 多个烟幕弹爆炸时间列表
    current_time -- 当前时间
    smoke_radius -- 烟幕云团半径,10m
    last_time -- 烟幕有效持续时间,20s

    返回:
    bool -- 是否被有效遮蔽
    """
    # 筛选有效的云团(已爆炸且未超过持续时间)
    active_clouds = []
    for cloud_center, explosion_time in zip(cloud_centers, explosion_times):
        if cloud_center is not None and current_time - explosion_time <= last_time:
            active_clouds.append(cloud_center)

    # 如果没有有效云团, 直接返回False
    if not active_clouds:
        return False

    # 检查每个目标点
    for target_point in target_points:
        # 导弹到目标点的视线向量
        sight_vector = np.array(target_point) - np.array(missile_pos)
        sight_length = np.linalg.norm(sight_vector)

        # 如果导弹已经到达目标点, 视为被遮蔽
        if sight_length < 1e-6:
            return True

        # 检查是否至少有一个云团遮蔽了视线
        line_shielded = False

        for cloud_center in active_clouds:
            # 云团中心到视线的投影比例
```

```
            mc_vector = np.array(cloud_center) - np.array(missile_pos)
            t = np.dot(mc_vector, sight_vector) / (sight_length**2)

            # 云团中心到视线的垂直距离
            distance = np.linalg.norm(mc_vector - t * sight_vector)

            # 云团是否在导弹和目标之间
            in_between = 0 <= t <= 1

            # 判断该云团是否遮蔽了视线
            if distance < smoke_radius and in_between:
                line_shielded = True
                break
            elif distance < smoke_radius:
                # 如果云团不在导弹和目标之间，检查是否足够接近目标或导弹
                if t > 1:  # 云团在目标"后方"
                    distance_to_target = np.linalg.norm(
                        np.array(cloud_center) - np.array(target_point)
                    )
                    if distance_to_target < smoke_radius:
                        line_shielded = True
                        break
                else:  # 云团在导弹"后方"(t<0)
                    distance_to_missile = np.linalg.norm(
                        np.array(cloud_center) - np.array(missile_pos)
                    )
                    if distance_to_missile < smoke_radius:
                        line_shielded = True
                        break

        # 如果所有云团都未能遮蔽该视线，则整体未被遮蔽
        if not line_shielded:
            return False

    # 所有视线都被遮蔽，则整体被遮蔽
    return True
```

```python


def calculate_missile_position(initial_pos, time, velocity=300, target_pos=(0, 0, 0)):
    """
    计算导弹在给定时间的位置

    参数:
    initial_pos -- 导弹初始位置 (x,y,z)
    time -- 时间(秒)
    velocity -- 导弹速度(米/秒),默认300
    target_pos -- 导弹目标位置(假目标), 默认(0,0,0)

    返回:
    tuple -- 导弹当前位置 (x,y,z)
    """
    initial_pos = np.array(initial_pos)
    target_pos = np.array(target_pos)

    direction = target_pos - initial_pos
    direction_unit = direction / np.linalg.norm(direction)
    velocity_vector = direction_unit * velocity

    new_pos = initial_pos + velocity_vector * time
    return tuple(new_pos)


def calculate_plane_position(initial_pos, direction, speed, time):
    """
    计算飞机在给定时间的位置

    参数:
    initial_pos -- 飞机初始位置 (x,y,z)
    direction -- 飞行方向向量
    speed -- 飞行速度(米/秒)
    time -- 时间(秒)

    返回:
    tuple -- 飞机当前位置 (x,y,z)
    """
```

```
    initial_pos = np.array(initial_pos)
    direction = np.array(direction)

    direction_unit = direction / np.linalg.norm(direction)
    velocity_vector = direction_unit * speed

    new_pos = initial_pos + velocity_vector * time
    return tuple(new_pos)


def calculate_bomb_position(
    plane_initial_pos,
    plane_direction,
    plane_speed,
    release_time,
    fall_time,
    gravity=9.8,
):
    """
    计算烟幕弹轨迹和爆炸位置

    参数:
    plane_initial_pos -- 无人机初始位置 (x,y,z)
    plane_direction -- 无人机飞行方向
    plane_speed -- 无人机速度(米/秒)
    release_time -- 投放时间(秒)
    fall_time -- 烟幕弹下落时间(秒)
    gravity -- 重力加速度,默认9.8

    返回:
    tuple -- 爆炸位置 (x,y,z)
    """
    # 计算投放位置
    release_position = calculate_plane_position(
        plane_initial_pos, plane_direction, plane_speed, release_time
    )

    # 计算水平位移
    plane_direction = np.array(plane_direction)
    direction_unit = plane_direction / np.linalg.norm(plane_direction
```

```
        )
    velocity_vector = direction_unit * plane_speed

    # 计算爆炸位置
    explosion_pos = (
        release_position[0] + velocity_vector[0] * fall_time,
        release_position[1] + velocity_vector[1] * fall_time,
        release_position[2] - 0.5 * gravity * fall_time**2,
    )

    return explosion_pos


def calculate_cloud_position(t, explosion_position, explosion_time,
    sink_speed=3):
    """
    计算特定时刻的云团位置

    参数:
    t -- 当前时间(秒)
    explosion_position -- 爆炸位置 (x,y,z)
    explosion_time -- 爆炸时间(秒)
    sink_speed -- 云团下沉速度(米/秒),默认3

    返回:
    list/None -- 云团位置 [x,y,z] 或 None(未爆炸)
    """
    # 云团匀速下沉
    if t < explosion_time:
        return None  # 未起爆
    else:
        return [
            explosion_position[0],
            explosion_position[1],
            explosion_position[2] - sink_speed * (t - explosion_time)
                ,
        ]


def generate_cylinder_sample_points(
```

```
    base_center=(0, 200, 0), height=10, radius=7, num_points=5000
):
    """
    生成圆柱体目标上的采样点

    参数:
    base_center -- 底面圆心坐标, 默认(0,200,0)
    height -- 圆柱体高度, 默认10
    radius -- 圆柱体半径, 默认7
    num_points -- 每个圆面上的采样点数, 默认100

    返回:
    list -- 圆柱体表面采样点列表
    """
    base_center = np.array(base_center)
    top_center = np.array([base_center[0], base_center[1],
        base_center[2] + height])

    points = []

    # 底面采样点
    for i in range(num_points):
        angle = 2 * np.pi * i / num_points
        x = base_center[0] + radius * np.cos(angle)
        y = base_center[1] + radius * np.sin(angle)
        z = base_center[2]
        points.append((x, y, z))

    # 顶面采样点
    for i in range(num_points):
        angle = 2 * np.pi * i / num_points
        x = top_center[0] + radius * np.cos(angle)
        y = top_center[1] + radius * np.sin(angle)
        z = top_center[2]
        points.append((x, y, z))

    return points


def calculate_shielding_duration(
```

```
    missile_initial_pos=(20000, 0, 2000),
    missile_velocity=300,
    missile_target=(0, 0, 0),
    plane_initial_pos=(17800, 0, 1800),
    plane_target=(0, 0, 1800),
    plane_speed=120,
    release_time=1.5,
    explosion_delay=3.6,
    smoke_radius=10,
    smoke_last_time=20,
    cloud_sink_speed=3,
    target_center=(0, 200, 0),
    target_height=10,
    target_radius=7,
    sample_points=5000,
    time_step=0.005,
    verbose=True,
):
    """
    计算导弹被烟幕有效遮蔽的总时长

    参数:
    missile_initial_pos -- 导弹初始位置, 默认(20000,0,2000)
    missile_velocity -- 导弹速度(米/秒), 默认300
    missile_target -- 导弹目标位置(假目标), 默认(0,0,0)
    plane_initial_pos -- 无人机初始位置, 默认(17800,0,1800)
    plane_target -- 无人机目标位置, 默认(0,0,0)
    plane_speed -- 无人机速度(米/秒), 默认120
    release_time -- 烟幕弹投放时间(秒), 默认1.5
    explosion_delay -- 烟幕弹起爆延迟(秒), 默认3.6
    smoke_radius -- 烟幕云团半径(米), 默认10
    smoke_last_time -- 烟幕有效持续时间(秒), 默认20
    cloud_sink_speed -- 云团下沉速度(米/秒), 默认3
    target_center -- 真目标底面圆心坐标, 默认(0,200,0)
    target_height -- 真目标高度(米), 默认10
    target_radius -- 真目标半径(米), 默认7
    sample_points -- 圆柱体每个面的采样点数, 默认100
    time_step -- 时间步长(秒), 默认0.01
    verbose -- 是否输出详细信息, 默认True

```

```
    返回:
    tuple -- (总遮蔽时长(秒), 遮蔽时间段列表[(开始时间,结束时间),...]
    """
    # 计算无人机飞行方向
    plane_direction = np.array(plane_target) - np.array(
        plane_initial_pos)

    # 计算爆炸位置和时间
    explosion_position = calculate_bomb_position(
        plane_initial_pos, plane_direction, plane_speed, release_time
            , explosion_delay
    )

    explosion_time = release_time + explosion_delay

    # 生成目标采样点
    target_points = generate_cylinder_sample_points(
        target_center, target_height, target_radius, sample_points
    )

    if verbose:
        print("=== 烟幕干扰分析 ===")
        print(f"无人机初始位置: {plane_initial_pos}")
        print(f"飞行速度: {plane_speed} m/s, 朝向目标: {plane_target}
            ")
        print(f"投放时间: {release_time}s")
        print(f"起爆延时: {explosion_delay}s")
        print(f"爆炸时间: {explosion_time}s")
        print(
            f"爆炸位置: ({explosion_position[0]:.1f}, {
                explosion_position[1]:.1f}, {explosion_position[2]:.1f
                })"
        )
        print(f"烟幕半径: {smoke_radius}m")
        print(f"烟幕有效时间: {smoke_last_time}s")
        print(f"云团下沉速度: {cloud_sink_speed} m/s")
        print(f"\n目标信息:")
        print(
            f"真目标位置: 圆心{target_center}, 半径{target_radius}m,
                高{target_height}m"
```

```
        )
        print(f"采样点数: {len(target_points)}")
        print(f"\n开始模拟...")

    # 遍历时间步长
    time_start = 0
    time_end = 100  # 足够长的时间范围

    shielded_times = []
    is_currently_shielded = False
    shield_start_time = None

    # 记录关键时刻信息
    first_shield_info = None
    last_shield_info = None

    for t in np.arange(time_start, time_end, time_step):
        # 计算当前位置
        missile_pos = calculate_missile_position(
            missile_initial_pos, t, missile_velocity, missile_target
        )
        cloud_pos = calculate_cloud_position(
            t, explosion_position, explosion_time, cloud_sink_speed
        )

        # 检查导弹是否已经通过目标区域
        missile_pos_array = np.array(missile_pos)
        if np.linalg.norm(missile_pos_array) < 1:
            if is_currently_shielded:
                shielded_times.append((shield_start_time, t))
                last_shield_info = {
                    "time": t,
                    "missile_pos": missile_pos,
                    "cloud_pos": cloud_pos,
                    "missile_distance": np.linalg.norm(
                        missile_pos_array - np.array(target_center)
                    ),
                }
            break

```

```
        # 检查云团是否已形成
        if cloud_pos is None:
            continue

        # 判断是否有效遮蔽
        is_shielded = is_effectively_shielded(
            missile_pos,
            target_points,
            cloud_pos,
            explosion_time,
            t,
            smoke_radius,
            smoke_last_time,
        )

        # 记录遮蔽时间段
        if is_shielded and not is_currently_shielded:
            is_currently_shielded = True
            shield_start_time = t
            if first_shield_info is None:
                first_shield_info = {
                    "time": t,
                    "missile_pos": missile_pos,
                    "cloud_pos": cloud_pos,
                    "missile_distance": np.linalg.norm(
                        np.array(missile_pos) - np.array(
                            target_center)
                    ),
                }
        elif not is_shielded and is_currently_shielded:
            is_currently_shielded = False
            shielded_times.append((shield_start_time, t))
            last_shield_info = {
                "time": t,
                "missile_pos": missile_pos,
                "cloud_pos": cloud_pos,
                "missile_distance": np.linalg.norm(
                    np.array(missile_pos) - np.array(target_center)
                ),
            }
```

```python

    # 输出详细信息
    if verbose:
        print(f"\n=== 遮蔽时间段详情 ===")
        if len(shielded_times) == 0:
            print("未检测到有效遮蔽！")
            # 调试信息
            t_test = 10.0  # 测试时刻
            missile_test = calculate_missile_position(
                missile_initial_pos, t_test, missile_velocity, missile_target
            )
            cloud_test = calculate_cloud_position(
                t_test, explosion_position, explosion_time, cloud_sink_speed
            )
            if cloud_test:
                print(f"\n调试信息:")
                print(f"t={t_test}s时:")
                print(f"  导弹位置: {missile_test}")
                print(f"  云团位置: {cloud_test}")
                print(
                    f"  导弹到真目标距离: {np.linalg.norm(np.array(missile_test) - np.array(target_center)):.1f}m"
                )
        else:
            for i, (start, end) in enumerate(shielded_times):
                print(f"第{i+1}段: {start:.2f}s - {end:.2f}s (持续 {end-start:.2f}s)")

            if first_shield_info:
                print(f"\n首次遮蔽时刻 (t={first_shield_info['time']:.2f}s):")
                print(f"  导弹位置: {first_shield_info['missile_pos']}")
                print(f"  云团位置: {first_shield_info['cloud_pos']}")
                print(f"  导弹距真目标: {first_shield_info['missile_distance']:.1f}m")
```

```python

            if last_shield_info:
                print(f"\n最后遮蔽时刻 (t={last_shield_info['time']:.2f}s):")
                print(f"  导弹位置: {last_shield_info['missile_pos']}")
                print(f"  云团位置: {last_shield_info['cloud_pos']}")
                print(f"  导弹距真目标: {last_shield_info['missile_distance']:.1f}m")

        # 计算总遮蔽时长
        total_duration = sum(end - start for start, end in shielded_times)
        print(f"\n=== 结果汇总 ===")
        print(f"遮蔽段数: {len(shielded_times)}")
        print(f"总遮蔽时长: {total_duration:.4f} 秒")

    return sum(end - start for start, end in shielded_times), shielded_times


fitness_cache = {}


def position_to_key(pos):
    """将位置转换为缓存用的键"""
    return tuple(float(f"{x:.4f}") for x in pos)


def evaluate_position(position):
    """评估一个位置(解)的适应度"""
    key = position_to_key(position)
    if key in fitness_cache:
        return fitness_cache[key]

    # 解析参数
    (
        azimuth,
        speed,
        release_time1,
```

```
        explosion_delay1,
        release_time2,
        explosion_delay2,
        release_time3,
        explosion_delay3,
    ) = position

    # 计算飞行方向向量
    direction_x = np.cos(azimuth)
    direction_y = np.sin(azimuth)
    direction_z = 0  # 水平飞行

    plane_direction = (direction_x, direction_y, direction_z)

    # 计算目标位置（保持z坐标不变）
    dir_norm = np.sqrt(direction_x**2 + direction_y**2)
    if dir_norm < 1e-6:  # 避免除零
        return 0.0

    target_distance = 20000  # 足够远的距离
    plane_target = (
        17800 + direction_x / dir_norm * target_distance,
        0 + direction_y / dir_norm * target_distance,
        1800,  # 保持z坐标不变
    )

    # 计算三枚烟幕弹的遮蔽区间
    total_shielding_periods = []

    # 三枚烟幕弹的计算
    for release_time, explosion_delay in [
        (release_time1, explosion_delay1),
        (release_time2, explosion_delay2),
        (release_time3, explosion_delay3),
    ]:
        try:
            _, shield_times = calculate_shielding_duration(
                missile_initial_pos=(20000, 0, 2000),
                missile_velocity=300,
                missile_target=(0, 0, 0),
```

```
                plane_initial_pos=(17800, 0, 1800),
                plane_target=plane_target,
                plane_speed=speed,
                release_time=release_time,
                explosion_delay=explosion_delay,
                sample_points=100,  # 降低精度加快评估
                time_step=0.02,
                verbose=False,
            )
            total_shielding_periods.extend(shield_times)
        except Exception:
            pass

    # 如果没有有效遮蔽，返回0
    if not total_shielding_periods:
        result = 0.0
        fitness_cache[key] = result
        return result

    # 合并重叠的遮蔽时间段
    total_shielding_periods.sort(key=lambda x: x[0])
    merged_periods = [total_shielding_periods[0]]

    for current in total_shielding_periods[1:]:
        previous = merged_periods[-1]
        if current[0] <= previous[1]:
            # 有重叠，合并区间
            merged_periods[-1] = (previous[0], max(previous[1],
                current[1]))
        else:
            # 无重叠，添加新区间
            merged_periods.append(current)

    # 计算总遮蔽时长
    total_duration = sum(end - start for start, end in merged_periods
        )

    # 添加奖励 - 奖励更均匀分布的投放时间
    time_gaps = [release_time2 - release_time1, release_time3 -
        release_time2]
```

```
    if min(time_gaps) >= 1.0 and max(time_gaps) <= 5.0:
        # 投放间隔在1-5秒之间，奖励均衡的投放策略
        total_duration *= 1.1

    fitness_cache[key] = total_duration
    return total_duration


def optimize_with_pso_three_bombs():
    """使用PSO优化一架无人机投放三枚烟幕弹的策略"""
    start_time = time.time()

    # 定义FY1和M1的初始位置
    FY1_initial_pos = (17800, 0, 1800)
    M1_initial_pos = (20000, 0, 2000)

    # 定义参数边界 - 8个参数
    bounds = [
        (-np.pi / 10, np.pi / 10),  # 方位角
        (80, 140),  # 飞行速度
        (0, 4),  # 第一枚投放时间
        (0, 1),  # 第一枚延迟
        (0, 4),  # 第二枚投放时间
        (0, 1),  # 第二枚延迟
        (0, 4),  # 第三枚投放时间
        (0, 1),  # 第三枚延迟
    ]

    # 创建缓存
    manager = multiprocessing.Manager()
    fitness_cache = manager.dict()

    # 创建粒子类
    class Particle:
        def __init__(self, dimensions):
            # 初始化位置和速度
            self.position = np.array(
                [random.uniform(low, high) for low, high in bounds]
            )
            self.velocity = np.array(
```

```
                [
                    random.uniform(-0.5 * (high - low), 0.5 * (high - low))
                    for low, high in bounds
                ]
            )

            # 调整投放时间顺序
            self._adjust_release_times()

            # 初始化粒子的最佳位置和适应度
            self.best_position = self.position.copy()
            self.best_fitness = -float("inf")
            self.fitness = -float("inf")

        def _adjust_release_times(self):
            """确保投放时间顺序: t1 < t2 < t3"""
            # 确保第二枚比第一枚晚至少1秒
            if self.position[4] <= self.position[2] + 1:
                self.position[4] = self.position[2] + 1

            # 确保第三枚比第二枚晚至少1秒
            if self.position[6] <= self.position[4] + 1:
                self.position[6] = self.position[4] + 1

        def update_velocity(self, global_best_position, w=0.7, c1=1.5, c2=1.5):
            """更新粒子的速度"""
            r1 = np.random.rand(len(self.position))
            r2 = np.random.rand(len(self.position))

            cognitive_component = c1 * r1 * (self.best_position - self.position)
            social_component = c2 * r2 * (global_best_position - self.position)

            self.velocity = w * self.velocity + cognitive_component + social_component

        def update_position(self):
```

```
        """更新粒子的位置并确保在边界内"""
        self.position = self.position + self.velocity

        # 边界处理
        for i in range(len(self.position)):
            if self.position[i] < bounds[i][0]:
                self.position[i] = bounds[i][0]
                self.velocity[i] *= -0.5  # 反弹
            elif self.position[i] > bounds[i][1]:
                self.position[i] = bounds[i][1]
                self.velocity[i] *= -0.5  # 反弹

        # 调整投放时间顺序
        self._adjust_release_times()

# PSO参数
num_particles = 300
max_iterations = 50

# 自适应惯性权重
w_start = 0.9  # 初始较大的惯性权重促进全局搜索
w_end = 0.4  # 最终较小的惯性权重促进局部精细搜索

# 创建粒子群
particles = [Particle(len(bounds)) for _ in range(num_particles)]

# 初始化全局最佳
global_best_position = None
global_best_fitness = -float("inf")

# 设置并行池
pool = multiprocessing.Pool(processes=multiprocessing.cpu_count()
    )

# 创建历史记录
history = {"best_fitness": [], "avg_fitness": []}

try:
    print(f"使用PSO优化，粒子数量={num_particles}，最大迭代={
        max_iterations}")
```

```
        print(f"使用{multiprocessing.cpu_count()}个CPU核心进行并行计算")

        # 初始评估
        fitnesses = pool.map(evaluate_position, [p.position for p in particles])

        for i, fitness in enumerate(fitnesses):
            particles[i].fitness = fitness
            particles[i].best_fitness = fitness
            particles[i].best_position = particles[i].position.copy()

            if fitness > global_best_fitness:
                global_best_fitness = fitness
                global_best_position = particles[i].position.copy()

        # PSO主循环
        stagnation_counter = 0
        prev_best = global_best_fitness

        for iteration in range(max_iterations):
            # 计算当前的惯性权重（线性递减）
            w = w_start - (w_start - w_end) * iteration / max_iterations

            # 更新所有粒子
            for p in particles:
                p.update_velocity(global_best_position, w=w)
                p.update_position()

            # 并行评估
            fitnesses = pool.map(evaluate_position, [p.position for p in particles])

            for i, fitness in enumerate(fitnesses):
                particles[i].fitness = fitness

                # 更新个体最佳
                if fitness > particles[i].best_fitness:
                    particles[i].best_fitness = fitness
```

```
                particles[i].best_position = particles[i].position.copy()

            # 更新全局最佳
            if fitness > global_best_fitness:
                global_best_fitness = fitness
                global_best_position = particles[i].position.copy()
                stagnation_counter = 0

        # 检测停滞并应用扰动
        if abs(global_best_fitness - prev_best) < 0.001:
            stagnation_counter += 1
        else:
            stagnation_counter = 0
            prev_best = global_best_fitness

        if stagnation_counter >= 10:
            print(f"迭代 {iteration}: 检测到停滞, 应用粒子重置")

            # 重置40%的粒子
            reset_count = int(0.4 * num_particles)
            indices = np.argsort([p.fitness for p in particles])[:reset_count]

            for idx in indices:
                if random.random() < 0.7:
                    # 随机重置
                    particles[idx] = Particle(len(bounds))
                else:
                    # 基于全局最佳的变异重置
                    new_position = global_best_position + np.random.normal(
                        0, 0.2, size=len(bounds)
                    ) * np.array([(high - low) for low, high in bounds])
                    # 确保新位置在边界内
                    for j in range(len(new_position)):
                        new_position[j] = min(
                            max(new_position[j], bounds[j][0]),
```

```
                                bounds[j][1]
                            )

                        # 创建新粒子并替换
                        new_particle = Particle(len(bounds))
                        new_particle.position = new_position
                        particles[idx] = new_particle

                stagnation_counter = 0

            # 收集历史数据
            current_avg = sum(p.fitness for p in particles) / len(
                particles)
            history["best_fitness"].append(global_best_fitness)
            history["avg_fitness"].append(current_avg)

            # 打印进度
            if iteration % 5 == 0 or iteration == max_iterations - 1:
                print(
                    f"迭代 {iteration}: 最佳适应度 = {
                        global_best_fitness:.4f}, 平均适应度 = {
                        current_avg:.4f}, 缓存数 = {len(fitness_cache)
                        }"
                )

        print("\nPSO优化完成!")
        print(f"最佳适应度值: {global_best_fitness:.4f}")

        # 解析最佳位置参数
        (
            azimuth,
            speed,
            release_time1,
            explosion_delay1,
            release_time2,
            explosion_delay2,
            release_time3,
            explosion_delay3,
        ) = global_best_position

```

```python
        # 计算飞行方向
        direction_x = np.cos(azimuth)
        direction_y = np.sin(azimuth)
        direction_z = 0
        direction = (direction_x, direction_y, direction_z)

        # 计算目标位置
        dir_norm = np.sqrt(direction_x**2 + direction_y**2)
        target_distance = 20000
        plane_target = (
            FY1_initial_pos[0] + direction_x / dir_norm *
               target_distance,
            FY1_initial_pos[1] + direction_y / dir_norm *
               target_distance,
            FY1_initial_pos[2],
        )

        # 转换为角度 (0-360度)
        direction_angle = (np.arctan2(direction_y, direction_x) * 180
            / np.pi) % 360

        # 计算三枚烟幕弹的详细参数
        bombs_data = []
        for idx, (release_time, explosion_delay) in enumerate(
            [
                (release_time1, explosion_delay1),
                (release_time2, explosion_delay2),
                (release_time3, explosion_delay3),
            ]
        ):
            # 计算投放点和起爆点
            release_position = calculate_plane_position(
                FY1_initial_pos, direction, speed, release_time
            )

            explosion_position = calculate_bomb_position(
                FY1_initial_pos, direction, speed, release_time,
                   explosion_delay
            )

```

```
        # 计算单枚干扰时长
        duration, _ = calculate_shielding_duration(
            missile_initial_pos=M1_initial_pos,
            missile_velocity=300,
            missile_target=(0, 0, 0),
            plane_initial_pos=FY1_initial_pos,
            plane_target=plane_target,
            plane_speed=speed,
            release_time=release_time,
            explosion_delay=explosion_delay,
            sample_points=300,
            verbose=False,
        )

        bombs_data.append(
            {
                "编号": idx + 1,
                "运动方向": direction_angle,
                "运动速度": speed,
                "投放点x": release_position[0],
                "投放点y": release_position[1],
                "投放点z": release_position[2],
                "起爆点x": explosion_position[0],
                "起爆点y": explosion_position[1],
                "起爆点z": explosion_position[2],
                "有效干扰时长": duration,
            }
        )

    # 计算合并后的总有效干扰时长
    total_shielding_periods = []
    for release_time, explosion_delay in [
        (release_time1, explosion_delay1),
        (release_time2, explosion_delay2),
        (release_time3, explosion_delay3),
    ]:
        _, shield_times = calculate_shielding_duration(
            missile_initial_pos=M1_initial_pos,
            missile_velocity=300,
            missile_target=(0, 0, 0),
```

```
                plane_initial_pos=FY1_initial_pos,
                plane_target=plane_target,
                plane_speed=speed,
                release_time=release_time,
                explosion_delay=explosion_delay,
                sample_points=300,
                verbose=False,
            )
            total_shielding_periods.extend(shield_times)

        # 合并重叠的遮蔽时间段
        if total_shielding_periods:
            total_shielding_periods.sort(key=lambda x: x[0])
            merged_periods = [total_shielding_periods[0]]

            for current in total_shielding_periods[1:]:
                previous = merged_periods[-1]
                if current[0] <= previous[1]:
                    # 有重叠，合并区间
                    merged_periods[-1] = (previous[0], max(previous[1], current[1]))
                else:
                    # 无重叠，添加新区间
                    merged_periods.append(current)

            total_duration = sum(end - start for start, end in merged_periods)
        else:
            total_duration = 0.0
            merged_periods = []

        # 创建结果DataFrame
        df = pd.DataFrame(
            columns=[
                "无人机运动方向",
                "无人机运动速度 (m/s)",
                "烟幕干扰弹编号",
                "烟幕干扰弹投放点的x坐标 (m)",
                "烟幕干扰弹投放点的y坐标 (m)",
                "烟幕干扰弹投放点的z坐标 (m)",
```

```
            "烟幕干扰弹起爆点的x坐标 (m)",
            "烟幕干扰弹起爆点的y坐标 (m)",
            "烟幕干扰弹起爆点的z坐标 (m)",
            "有效干扰时长 (s)",
        ]
    )

    # 填充数据
    for i, bomb in enumerate(bombs_data):
        df.loc[i] = [
            bomb["运动方向"],
            bomb["运动速度"],
            bomb["编号"],
            bomb["投放点x"],
            bomb["投放点y"],
            bomb["投放点z"],
            bomb["起爆点x"],
            bomb["起爆点y"],
            bomb["起爆点z"],
            bomb["有效干扰时长"],
        ]

    # 打印优化结果
    print("\n=== 优化结果 ===")
    print(f"无人机运动方向: {direction_angle:.2f}度")
    print(f"无人机运动速度: {speed:.2f} m/s")
    print(f"总有效干扰时长: {total_duration:.2f}秒")
    print(
        f"烟幕弹1: 投放时间={release_time1:.2f}s, 延迟={explosion_delay1:.2f}s, 干扰时长={bombs_data[0]['有效干扰时长']:.2f}s"
    )
    print(
        f"烟幕弹2: 投放时间={release_time2:.2f}s, 延迟={explosion_delay2:.2f}s, 干扰时长={bombs_data[1]['有效干扰时长']:.2f}s"
    )
    print(
        f"烟幕弹3: 投放时间={release_time3:.2f}s, 延迟={explosion_delay3:.2f}s, 干扰时长={bombs_data[2]['有效
```

```
            干扰时长']:.2f}s"
        )

        # 绘制优化过程图
        plt.figure(figsize=(12, 6))
        plt.plot(
            range(max_iterations),
            history["best_fitness"],
            label="最佳适应度",
            linewidth=2,
        )
        plt.plot(
            range(max_iterations),
            history["avg_fitness"],
            label="平均适应度",
            linewidth=1,
            alpha=0.7,
        )
        plt.xlabel("迭代次数")
        plt.ylabel("适应度(总遮蔽时长)")
        plt.title("PSO优化过程 - 问题三")
        plt.legend()
        plt.grid(True)
        plt.savefig("PSO优化进程_问题三.png", dpi=300)
        plt.show()

        # 分析遮蔽时间段
        print("\n=== 遮蔽时间段分析 ===")
        print(f"共有{len(merged_periods)}个遮蔽时间段:")
        for i, (start, end) in enumerate(merged_periods):
            print(f"段{i+1}: {start:.2f}s - {end:.2f}s (持续{end-start:.2f}s)")

        end_time = time.time()
        print(f"\nPSO优化总耗时: {end_time - start_time:.2f}秒")

        return global_best_position, bombs_data, total_duration, merged_periods

    finally:
```

```python
        # 关闭进程池
        pool.close()
        pool.join()


def run_pso_three_bombs():
    """运行PSO优化三枚烟幕弹投放策略"""

    print("开始粒子群优化一架无人机投放三枚烟幕弹策略...")

    try:
        # 运行优化
        best_position, bombs_data, total_duration, shield_periods = (
            optimize_with_pso_three_bombs()
        )
        print(f"\n优化成功完成! 最大总遮蔽时长: {total_duration:.4f}秒")

    except Exception as e:
        print(f"优化过程中出错: {e}")
        import traceback

        traceback.print_exc()


if __name__ == "__main__":
    run_pso_three_bombs()




import numpy as np
import random
import matplotlib.pyplot as plt
import pandas as pd
import time
import multiprocessing
```

```python

import numpy as np


def is_effectively_shielded(
    missile_pos,
    target_points,
    cloud_center,
    explosion_time,
    current_time,
    smoke_radius=10,
    last_time=20,
):
    """
    判断导弹是否被有效遮蔽

    参数:
    missile_pos -- 导弹当前位置坐标 (x,y,z)
    target_points -- 目标上的采样点列表
    cloud_center -- 烟幕云团中心位置 (x,y,z)
    explosion_time -- 烟幕弹爆炸时间
    current_time -- 当前时间
    smoke_radius -- 烟幕云团半径,10m
    last_time -- 烟幕有效持续时间,20s

    返回:
    bool -- 是否被有效遮蔽
    """
    # 时间检查
    if current_time - explosion_time > last_time:
        return False

    # 位置检查
    for target_point in target_points:
        # 导弹到目标点的视线向量
        sight_vector = np.array(target_point) - np.array(missile_pos)
        sight_length = np.linalg.norm(sight_vector)
        if sight_length < 1e-6:  # 避免除零
            return True

```

```
        # 云团中心到视线的投影比例
        mc_vector = np.array(cloud_center) - np.array(missile_pos)
        t = np.dot(mc_vector, sight_vector) / (sight_length**2)

        # 云团中心到视线的垂直距离
        distance = np.linalg.norm(mc_vector - t * sight_vector)

        # 云团是否在导弹和目标之间
        in_between = 0 <= t <= 1

        # 如果视线未被遮蔽或云团不在中间，则整体未被遮蔽
        if distance >= smoke_radius:
            # 如果云团离视线太远，肯定不遮蔽
            return False
        elif not in_between:
            # 如果云团不在导弹和目标之间，检查是否足够接近目标
            if t > 1:  # 云团在目标"后方"
                distance_to_target = np.linalg.norm(
                    np.array(cloud_center) - np.array(target_point)
                )
                return distance_to_target < smoke_radius
            else:  # 云团在导弹"后方"(t<0)
                distance_to_missile = np.linalg.norm(
                    np.array(cloud_center) - np.array(missile_pos)
                )
                return distance_to_missile < smoke_radius
    return True


def is_effectively_shielded_by_multiple_clouds(
    missile_pos,
    target_points,
    cloud_centers,
    explosion_times,
    current_time,
    smoke_radius=10,
    last_time=20,
):
    """
    判断导弹是否被多个烟幕云团有效遮蔽
```

```
    参数:
    missile_pos -- 导弹当前位置坐标 (x,y,z)
    target_points -- 目标上的采样点列表
    cloud_centers -- 多个烟幕云团中心位置列表, 每个元素为 (x,y,z) 或 None
    explosion_times -- 多个烟幕弹爆炸时间列表
    current_time -- 当前时间
    smoke_radius -- 烟幕云团半径,10m
    last_time -- 烟幕有效持续时间,20s

    返回:
    bool -- 是否被有效遮蔽
    """
    # 筛选有效的云团(已爆炸且未超过持续时间)
    active_clouds = []
    for cloud_center, explosion_time in zip(cloud_centers, explosion_times):
        if cloud_center is not None and current_time - explosion_time <= last_time:
            active_clouds.append(cloud_center)

    # 如果没有有效云团, 直接返回False
    if not active_clouds:
        return False

    # 检查每个目标点
    for target_point in target_points:
        # 导弹到目标点的视线向量
        sight_vector = np.array(target_point) - np.array(missile_pos)
        sight_length = np.linalg.norm(sight_vector)

        # 如果导弹已经到达目标点, 视为被遮蔽
        if sight_length < 1e-6:
            return True

        # 检查是否至少有一个云团遮蔽了视线
        line_shielded = False

        for cloud_center in active_clouds:
```

```
            # 云团中心到视线的投影比例
            mc_vector = np.array(cloud_center) - np.array(missile_pos
                )
            t = np.dot(mc_vector, sight_vector) / (sight_length**2)

            # 云团中心到视线的垂直距离
            distance = np.linalg.norm(mc_vector - t * sight_vector)

            # 云团是否在导弹和目标之间
            in_between = 0 <= t <= 1

            # 判断该云团是否遮蔽了视线
            if distance < smoke_radius and in_between:
                line_shielded = True
                break
            elif distance < smoke_radius:
                # 如果云团不在导弹和目标之间，检查是否足够接近目标或
                    导弹
                if t > 1:  # 云团在目标"后方"
                    distance_to_target = np.linalg.norm(
                        np.array(cloud_center) - np.array(
                            target_point)
                    )
                    if distance_to_target < smoke_radius:
                        line_shielded = True
                        break
                else:  # 云团在导弹"后方"(t<0)
                    distance_to_missile = np.linalg.norm(
                        np.array(cloud_center) - np.array(missile_pos
                            )
                    )
                    if distance_to_missile < smoke_radius:
                        line_shielded = True
                        break

        # 如果所有云团都未能遮蔽该视线，则整体未被遮蔽
        if not line_shielded:
            return False

    # 所有视线都被遮蔽，则整体被遮蔽
```

```
    return True


def calculate_missile_position(initial_pos, time, velocity=300,
    target_pos=(0, 0, 0)):
    """
    计算导弹在给定时间的位置

    参数:
    initial_pos -- 导弹初始位置 (x,y,z)
    time -- 时间(秒)
    velocity -- 导弹速度(米/秒),默认300
    target_pos -- 导弹目标位置(假目标), 默认(0,0,0)

    返回:
    tuple -- 导弹当前位置 (x,y,z)
    """
    initial_pos = np.array(initial_pos)
    target_pos = np.array(target_pos)

    direction = target_pos - initial_pos
    direction_unit = direction / np.linalg.norm(direction)
    velocity_vector = direction_unit * velocity

    new_pos = initial_pos + velocity_vector * time
    return tuple(new_pos)


def calculate_plane_position(initial_pos, direction, speed, time):
    """
    计算飞机在给定时间的位置

    参数:
    initial_pos -- 飞机初始位置 (x,y,z)
    direction -- 飞行方向向量
    speed -- 飞行速度(米/秒)
    time -- 时间(秒)

    返回:
    tuple -- 飞机当前位置 (x,y,z)
```

```
    """
    initial_pos = np.array(initial_pos)
    direction = np.array(direction)

    direction_unit = direction / np.linalg.norm(direction)
    velocity_vector = direction_unit * speed

    new_pos = initial_pos + velocity_vector * time
    return tuple(new_pos)


def calculate_bomb_position(
    plane_initial_pos,
    plane_direction,
    plane_speed,
    release_time,
    fall_time,
    gravity=9.8,
):
    """
    计算烟幕弹轨迹和爆炸位置

    参数:
    plane_initial_pos -- 无人机初始位置 (x,y,z)
    plane_direction -- 无人机飞行方向
    plane_speed -- 无人机速度(米/秒)
    release_time -- 投放时间(秒)
    fall_time -- 烟幕弹下落时间(秒)
    gravity -- 重力加速度,默认9.8

    返回:
    tuple -- 爆炸位置 (x,y,z)
    """
    # 计算投放位置
    release_position = calculate_plane_position(
        plane_initial_pos, plane_direction, plane_speed, release_time
    )

    # 计算水平位移
    plane_direction = np.array(plane_direction)
```

```
    direction_unit = plane_direction / np.linalg.norm(plane_direction)
    velocity_vector = direction_unit * plane_speed

    # 计算爆炸位置
    explosion_pos = (
        release_position[0] + velocity_vector[0] * fall_time,
        release_position[1] + velocity_vector[1] * fall_time,
        release_position[2] - 0.5 * gravity * fall_time**2,
    )

    return explosion_pos


def calculate_cloud_position(t, explosion_position, explosion_time, sink_speed=3):
    """
    计算特定时刻的云团位置

    参数:
    t -- 当前时间(秒)
    explosion_position -- 爆炸位置 (x,y,z)
    explosion_time -- 爆炸时间(秒)
    sink_speed -- 云团下沉速度(米/秒),默认3

    返回:
    list/None -- 云团位置 [x,y,z] 或 None(未爆炸)
    """
    # 云团匀速下沉
    if t < explosion_time:
        return None  # 未起爆
    else:
        return [
            explosion_position[0],
            explosion_position[1],
            explosion_position[2] - sink_speed * (t - explosion_time),
        ]


```

```python
def generate_cylinder_sample_points(
    base_center=(0, 200, 0), height=10, radius=7, num_points=5000
):
    """
    生成圆柱体目标上的采样点

    参数:
    base_center -- 底面圆心坐标, 默认(0,200,0)
    height -- 圆柱体高度, 默认10
    radius -- 圆柱体半径, 默认7
    num_points -- 每个圆面上的采样点数, 默认100

    返回:
    list -- 圆柱体表面采样点列表
    """
    base_center = np.array(base_center)
    top_center = np.array([base_center[0], base_center[1],
        base_center[2] + height])

    points = []

    # 底面采样点
    for i in range(num_points):
        angle = 2 * np.pi * i / num_points
        x = base_center[0] + radius * np.cos(angle)
        y = base_center[1] + radius * np.sin(angle)
        z = base_center[2]
        points.append((x, y, z))

    # 顶面采样点
    for i in range(num_points):
        angle = 2 * np.pi * i / num_points
        x = top_center[0] + radius * np.cos(angle)
        y = top_center[1] + radius * np.sin(angle)
        z = top_center[2]
        points.append((x, y, z))

    return points
```

```python
def calculate_shielding_duration(
    missile_initial_pos=(20000, 0, 2000),
    missile_velocity=300,
    missile_target=(0, 0, 0),
    plane_initial_pos=(17800, 0, 1800),
    plane_target=(0, 0, 1800),
    plane_speed=120,
    release_time=1.5,
    explosion_delay=3.6,
    smoke_radius=10,
    smoke_last_time=20,
    cloud_sink_speed=3,
    target_center=(0, 200, 0),
    target_height=10,
    target_radius=7,
    sample_points=5000,
    time_step=0.005,
    verbose=True,
):
    """
    计算导弹被烟幕有效遮蔽的总时长

    参数:
    missile_initial_pos -- 导弹初始位置, 默认(20000,0,2000)
    missile_velocity -- 导弹速度(米/秒), 默认300
    missile_target -- 导弹目标位置(假目标), 默认(0,0,0)
    plane_initial_pos -- 无人机初始位置, 默认(17800,0,1800)
    plane_target -- 无人机目标位置, 默认(0,0,0)
    plane_speed -- 无人机速度(米/秒), 默认120
    release_time -- 烟幕弹投放时间(秒), 默认1.5
    explosion_delay -- 烟幕弹起爆延迟(秒), 默认3.6
    smoke_radius -- 烟幕云团半径(米), 默认10
    smoke_last_time -- 烟幕有效持续时间(秒), 默认20
    cloud_sink_speed -- 云团下沉速度(米/秒), 默认3
    target_center -- 真目标底面圆心坐标, 默认(0,200,0)
    target_height -- 真目标高度(米), 默认10
    target_radius -- 真目标半径(米), 默认7
    sample_points -- 圆柱体每个面的采样点数, 默认100
    time_step -- 时间步长(秒), 默认0.01
    verbose -- 是否输出详细信息, 默认True
```

```
    返回:
    tuple -- (总遮蔽时长(秒), 遮蔽时间段列表[(开始时间,结束时间),...]
    """
    # 计算无人机飞行方向
    plane_direction = np.array(plane_target) - np.array(
        plane_initial_pos)

    # 计算爆炸位置和时间
    explosion_position = calculate_bomb_position(
        plane_initial_pos, plane_direction, plane_speed, release_time
            , explosion_delay
    )

    explosion_time = release_time + explosion_delay

    # 生成目标采样点
    target_points = generate_cylinder_sample_points(
        target_center, target_height, target_radius, sample_points
    )

    if verbose:
        print("=== 烟幕干扰分析 ===")
        print(f"无人机初始位置: {plane_initial_pos}")
        print(f"飞行速度: {plane_speed} m/s, 朝向目标: {plane_target}
            ")
        print(f"投放时间: {release_time}s")
        print(f"起爆延时: {explosion_delay}s")
        print(f"爆炸时间: {explosion_time}s")
        print(
            f"爆炸位置: ({explosion_position[0]:.1f}, {
                explosion_position[1]:.1f}, {explosion_position[2]:.1f
                })"
        )
        print(f"烟幕半径: {smoke_radius}m")
        print(f"烟幕有效时间: {smoke_last_time}s")
        print(f"云团下沉速度: {cloud_sink_speed} m/s")
        print(f"\n目标信息:")
        print(
            f"真目标位置: 圆心{target_center}, 半径{target_radius}m,
```

```
                高{target_height}m"
        )
        print(f"采样点数: {len(target_points)}")
        print(f"\n开始模拟...")

    # 遍历时间步长
    time_start = 0
    time_end = 100  # 足够长的时间范围

    shielded_times = []
    is_currently_shielded = False
    shield_start_time = None

    # 记录关键时刻信息
    first_shield_info = None
    last_shield_info = None

    for t in np.arange(time_start, time_end, time_step):
        # 计算当前位置
        missile_pos = calculate_missile_position(
            missile_initial_pos, t, missile_velocity, missile_target
        )
        cloud_pos = calculate_cloud_position(
            t, explosion_position, explosion_time, cloud_sink_speed
        )

        # 检查导弹是否已经通过目标区域
        missile_pos_array = np.array(missile_pos)
        if np.linalg.norm(missile_pos_array) < 1:
            if is_currently_shielded:
                shielded_times.append((shield_start_time, t))
                last_shield_info = {
                    "time": t,
                    "missile_pos": missile_pos,
                    "cloud_pos": cloud_pos,
                    "missile_distance": np.linalg.norm(
                        missile_pos_array - np.array(target_center)
                    ),
                }
            break
```

```python

        # 检查云团是否已形成
        if cloud_pos is None:
            continue

        # 判断是否有效遮蔽
        is_shielded = is_effectively_shielded(
            missile_pos,
            target_points,
            cloud_pos,
            explosion_time,
            t,
            smoke_radius,
            smoke_last_time,
        )

        # 记录遮蔽时间段
        if is_shielded and not is_currently_shielded:
            is_currently_shielded = True
            shield_start_time = t
            if first_shield_info is None:
                first_shield_info = {
                    "time": t,
                    "missile_pos": missile_pos,
                    "cloud_pos": cloud_pos,
                    "missile_distance": np.linalg.norm(
                        np.array(missile_pos) - np.array(
                            target_center)
                    ),
                }
        elif not is_shielded and is_currently_shielded:
            is_currently_shielded = False
            shielded_times.append((shield_start_time, t))
            last_shield_info = {
                "time": t,
                "missile_pos": missile_pos,
                "cloud_pos": cloud_pos,
                "missile_distance": np.linalg.norm(
                    np.array(missile_pos) - np.array(target_center)
                ),
```

```
            }

    # 输出详细信息
    if verbose:
        print(f"\n=== 遮蔽时间段详情 ===")
        if len(shielded_times) == 0:
            print("未检测到有效遮蔽！")
            # 调试信息
            t_test = 10.0  # 测试时刻
            missile_test = calculate_missile_position(
                missile_initial_pos, t_test, missile_velocity, missile_target
            )
            cloud_test = calculate_cloud_position(
                t_test, explosion_position, explosion_time, cloud_sink_speed
            )
            if cloud_test:
                print(f"\n调试信息:")
                print(f"t={t_test}s时:")
                print(f"  导弹位置: {missile_test}")
                print(f"  云团位置: {cloud_test}")
                print(
                    f"  导弹到真目标距离: {np.linalg.norm(np.array(missile_test) - np.array(target_center)):.1f}m"
                )
        else:
            for i, (start, end) in enumerate(shielded_times):
                print(f"第{i+1}段: {start:.2f}s - {end:.2f}s (持续 {end-start:.2f}s)")

            if first_shield_info:
                print(f"\n首次遮蔽时刻 (t={first_shield_info['time']:.2f}s):")
                print(f"  导弹位置: {first_shield_info['missile_pos']}")
                print(f"  云团位置: {first_shield_info['cloud_pos']}")
                print(f"  导弹距真目标: {first_shield_info['
```

```
                    missile_distance']:.1f}m")

            if last_shield_info:
                print(f"\n最后遮蔽时刻 (t={last_shield_info['time
                    ']:.2f}s):")
                print(f"  导弹位置: {last_shield_info['missile_pos']}
                    ")
                print(f"  云团位置: {last_shield_info['cloud_pos']}")
                print(f"  导弹距真目标: {last_shield_info['
                    missile_distance']:.1f}m")

        # 计算总遮蔽时长
        total_duration = sum(end - start for start, end in
            shielded_times)
        print(f"\n=== 结果汇总 ===")
        print(f"遮蔽段数: {len(shielded_times)}")
        print(f"总遮蔽时长: {total_duration:.4f} 秒")

    return sum(end - start for start, end in shielded_times),
        shielded_times


# 定义无人机和导弹初始位置
FY1_initial_pos = (17800, 0, 1800)
FY2_initial_pos = (12000, 1400, 1400)
FY3_initial_pos = (6000, -3000, 700)
M1_initial_pos = (20000, 0, 2000)


fitness_cache = {}


def cached_evaluate(position):
    """带缓存的评估函数"""
    key = tuple(round(x, 4) for x in position)
    if key in fitness_cache:
        return fitness_cache[key]

    result = evaluate_position(position)
    fitness_cache[key] = result
```

```
    return result


class Particle:
    """粒子类，表示PSO中的一个粒子"""

    def __init__(self, dimensions, bounds):
        """初始化一个粒子"""
        # 初始化位置和速度
        self.position = np.array([random.uniform(low, high) for low, high in bounds])
        self.velocity = np.array(
            [
                random.uniform(-0.5 * (high - low), 0.5 * (high - low))
                for low, high in bounds
            ]
        )

        # 初始化粒子的最佳位置和适应度
        self.best_position = self.position.copy()
        self.best_fitness = -float("inf")

        self.fitness = -float("inf")
        self.bounds = bounds

    def update_velocity(self, global_best_position, w=0.7, c1=1.5, c2=1.5):
        """更新粒子的速度"""
        r1 = np.random.rand(len(self.position))
        r2 = np.random.rand(len(self.position))

        cognitive_component = c1 * r1 * (self.best_position - self.position)
        social_component = c2 * r2 * (global_best_position - self.position)

        self.velocity = w * self.velocity + cognitive_component + social_component

```

```python
    def update_position(self):
        """更新粒子的位置并确保在边界内"""
        self.position = self.position + self.velocity

        # 边界处理
        for i in range(len(self.position)):
            if self.position[i] < self.bounds[i][0]:
                self.position[i] = self.bounds[i][0]
                self.velocity[i] *= -0.5  # 反弹
            elif self.position[i] > self.bounds[i][1]:
                self.position[i] = self.bounds[i][1]
                self.velocity[i] *= -0.5  # 反弹


def evaluate_position(position):
    """评估一个位置(解)的适应度"""
    # 解析参数
    (
        azimuth1,
        speed1,
        release_time1,
        delay1,
        azimuth2,
        speed2,
        release_time2,
        delay2,
        azimuth3,
        speed3,
        release_time3,
        delay3,
    ) = position

    # 记录每架无人机的贡献
    drone_contributions = [False, False, False]
    total_shielding_periods = []

    # 处理三架无人机的遮蔽计算
    for drone_idx, params in enumerate(
        [
            (FY1_initial_pos, azimuth1, speed1, release_time1, delay1
```

```
                ),
            (FY2_initial_pos, azimuth2, speed2, release_time2, delay2
                ),
            (FY3_initial_pos, azimuth3, speed3, release_time3, delay3
                ),
        ]
    ):
        initial_pos, azimuth, speed, release_time, delay = params

        # 计算飞行方向
        direction_x = np.cos(azimuth)
        direction_y = np.sin(azimuth)
        direction_z = 0
        direction = (direction_x, direction_y, direction_z)

        # 计算目标位置
        dir_norm = np.sqrt(direction_x**2 + direction_y**2)
        if dir_norm < 1e-6:  # 避免除零
            continue

        target_distance = 20000
        plane_target = (
            initial_pos[0] + direction_x / dir_norm * target_distance
                ,
            initial_pos[1] + direction_y / dir_norm * target_distance
                ,
            initial_pos[2],
        )

        try:
            _, shield_times = calculate_shielding_duration(
                missile_initial_pos=M1_initial_pos,
                missile_velocity=300,
                missile_target=(0, 0, 0),
                plane_initial_pos=initial_pos,
                plane_target=plane_target,
                plane_speed=speed,
                release_time=release_time,
                explosion_delay=delay,
                sample_points=100,  # 降低精度加快评估
```

```
                time_step=0.02,
                verbose=False,
            )

            if shield_times:  # 如果产生了有效遮蔽
                drone_contributions[drone_idx] = True
                total_shielding_periods.extend(shield_times)
        except Exception:
            pass

    # 无有效遮蔽
    if not total_shielding_periods:
        return 0.0

    # 合并重叠区间
    total_shielding_periods.sort(key=lambda x: x[0])
    merged_periods = [total_shielding_periods[0]]

    for current in total_shielding_periods[1:]:
        previous = merged_periods[-1]
        if current[0] <= previous[1]:  # 重叠区间
            merged_periods[-1] = (previous[0], max(previous[1],
                current[1]))
        else:
            merged_periods.append(current)

    # 计算总时长
    total_duration = sum(end - start for start, end in merged_periods
        )

    # 添加协同奖励
    active_drones = sum(drone_contributions)
    if active_drones == 2:
        total_duration *= 1.2  # 两架无人机协同奖励20%
    elif active_drones == 3:
        total_duration *= 1.5  # 三架无人机协同奖励50%

    return total_duration
```

```python
def optimize_with_pso():
    """使用PSO优化三架无人机的烟幕弹投放策略"""
    start_time = time.time()

    # 定义参数的边界 - 更宽泛的范围
    bounds = [
        (0, np.pi / 18),  # FY1: azimuth
        (110, 140),  # FY1: speed
        (0, 4),  # FY1: release_time
        (0, 4),  # FY1: delay
        (-np.pi / 4, 0),  # FY2: azimuth
        (100, 140),  # FY2: speed
        (15, 18),  # FY2: release_time
        (0, 3),  # FY2: delay
        (np.pi / 3, np.pi / 2),  # FY3: azimuth
        (100, 140),  # FY3: speed
        (27, 32),  # FY3: release_time
        (0, 3),  # FY3: delay
    ]

    # PSO参数
    num_particles = 150
    max_iterations = 100

    # 自适应惯性权重
    w_start = 0.9  # 初始较大的惯性权重促进全局搜索
    w_end = 0.4  # 最终较小的惯性权重促进局部精细搜索

    # 创建粒子群
    particles = [Particle(len(bounds), bounds) for _ in range(num_particles)]

    # 初始化全局最佳
    global_best_position = None
    global_best_fitness = -float("inf")

    # 设置并行池
    pool = multiprocessing.Pool(processes=multiprocessing.cpu_count())

```

```
    # 创建历史记录
    history = {"best_fitness": [], "avg_fitness": []}

    try:
        # 初始评估
        fitnesses = pool.map(cached_evaluate, [p.position for p in
            particles])
        for i, fitness in enumerate(fitnesses):
            particles[i].fitness = fitness
            particles[i].best_fitness = fitness
            particles[i].best_position = particles[i].position.copy()

            if fitness > global_best_fitness:
                global_best_fitness = fitness
                global_best_position = particles[i].position.copy()

        # PSO主循环
        stagnation_counter = 0
        prev_best = global_best_fitness

        for iteration in range(max_iterations):
            # 计算当前的惯性权重（线性递减）
            w = w_start - (w_start - w_end) * iteration /
                max_iterations

            # 更新所有粒子
            for p in particles:
                p.update_velocity(global_best_position, w=w)
                p.update_position()

            # 并行评估
            fitnesses = pool.map(cached_evaluate, [p.position for p
                in particles])

            for i, fitness in enumerate(fitnesses):
                particles[i].fitness = fitness

                # 更新个体最佳
                if fitness > particles[i].best_fitness:
                    particles[i].best_fitness = fitness
```

```
                particles[i].best_position = particles[i].position.copy()

            # 更新全局最佳
            if fitness > global_best_fitness:
                global_best_fitness = fitness
                global_best_position = particles[i].position.copy()
                stagnation_counter = 0

        # 检测停滞并应用扰动
        if abs(global_best_fitness - prev_best) < 0.001:
            stagnation_counter += 1
        else:
            stagnation_counter = 0
            prev_best = global_best_fitness

        if stagnation_counter >= 10:
            print(f"迭代 {iteration}: 检测到停滞, 应用粒子重置")

            # 重置40%的粒子
            reset_count = int(0.4 * num_particles)
            indices = np.argsort([p.fitness for p in particles])[:reset_count]

            for idx in indices:
                if random.random() < 0.7:
                    # 随机重置
                    particles[idx] = Particle(len(bounds), bounds)
                else:
                    # 基于全局最佳的变异重置
                    new_position = global_best_position + np.random.normal(
                        0, 0.2, size=len(bounds)
                    ) * np.array([(high - low) for low, high in bounds])
                    # 确保新位置在边界内
                    for j in range(len(new_position)):
                        new_position[j] = min(
```

```
                                max(new_position[j], bounds[j][0]),
                                    bounds[j][1]
                            )

                        # 创建新粒子并替换
                        new_particle = Particle(len(bounds), bounds)
                        new_particle.position = new_position
                        particles[idx] = new_particle

                stagnation_counter = 0

            # 收集历史数据
            current_avg = sum(p.fitness for p in particles) / len(
                particles)
            history["best_fitness"].append(global_best_fitness)
            history["avg_fitness"].append(current_avg)

            # 打印进度
            if iteration % 5 == 0 or iteration == max_iterations - 1:
                print(
                    f"迭代 {iteration}: 最佳适应度 = {
                        global_best_fitness:.4f}, 平均适应度 = {
                        current_avg:.4f}, 缓存数 = {len(fitness_cache)
                        }"
                )

        print("\nPSO优化完成!")
        print(f"最佳适应度值: {global_best_fitness:.4f}")

        # 解析最佳位置参数
        (
            azimuth1,
            speed1,
            release_time1,
            delay1,
            azimuth2,
            speed2,
            release_time2,
            delay2,
            azimuth3,
```

```
            speed3,
            release_time3,
            delay3,
        ) = global_best_position

        # 计算三架无人机的详细数据
        drones_data = []

        # 计算FY1数据
        direction1_x = np.cos(azimuth1)
        direction1_y = np.sin(azimuth1)
        direction1_z = 0
        direction1 = (direction1_x, direction1_y, direction1_z)

        direction1_angle = (np.arctan2(direction1_y, direction1_x) *
           180 / np.pi) % 360

        dir1_norm = np.sqrt(direction1_x**2 + direction1_y**2)
        target_distance = 20000
        plane1_target = (
            FY1_initial_pos[0] + direction1_x / dir1_norm *
               target_distance,
            FY1_initial_pos[1] + direction1_y / dir1_norm *
               target_distance,
            FY1_initial_pos[2],
        )

        release_position1 = calculate_plane_position(
            FY1_initial_pos, direction1, speed1, release_time1
        )

        explosion_position1 = calculate_bomb_position(
            FY1_initial_pos, direction1, speed1, release_time1,
               delay1
        )

        # 高精度评估
        duration1, _ = calculate_shielding_duration(
            missile_initial_pos=M1_initial_pos,
            missile_velocity=300,
```

```
            missile_target=(0, 0, 0),
            plane_initial_pos=FY1_initial_pos,
            plane_target=plane1_target,
            plane_speed=speed1,
            release_time=release_time1,
            explosion_delay=delay1,
            sample_points=500,
            verbose=False,
        )

        drones_data.append(
            {
                "编号": "FY1",
                "运动方向": direction1_angle,
                "运动速度": speed1,
                "投放点x": release_position1[0],
                "投放点y": release_position1[1],
                "投放点z": release_position1[2],
                "起爆点x": explosion_position1[0],
                "起爆点y": explosion_position1[1],
                "起爆点z": explosion_position1[2],
                "有效干扰时长": duration1,
                "投放时间": release_time1,  # 添加投放时间
                "引爆时间": delay1,  # 添加引爆时间
            }
        )

        # 计算FY2数据
        direction2_x = np.cos(azimuth2)
        direction2_y = np.sin(azimuth2)
        direction2_z = 0
        direction2 = (direction2_x, direction2_y, direction2_z)

        direction2_angle = (np.arctan2(direction2_y, direction2_x) *
           180 / np.pi) % 360

        dir2_norm = np.sqrt(direction2_x**2 + direction2_y**2)
        plane2_target = (
            FY2_initial_pos[0] + direction2_x / dir2_norm *
               target_distance,
```

```
            FY2_initial_pos[1] + direction2_y / dir2_norm *
                target_distance,
            FY2_initial_pos[2],
        )

        release_position2 = calculate_plane_position(
            FY2_initial_pos, direction2, speed2, release_time2
        )

        explosion_position2 = calculate_bomb_position(
            FY2_initial_pos, direction2, speed2, release_time2,
                delay2
        )

        duration2, _ = calculate_shielding_duration(
            missile_initial_pos=M1_initial_pos,
            missile_velocity=300,
            missile_target=(0, 0, 0),
            plane_initial_pos=FY2_initial_pos,
            plane_target=plane2_target,
            plane_speed=speed2,
            release_time=release_time2,
            explosion_delay=delay2,
            sample_points=500,
            verbose=False,
        )

        drones_data.append(
            {
                "编号": "FY2",
                "运动方向": direction2_angle,
                "运动速度": speed2,
                "投放点x": release_position2[0],
                "投放点y": release_position2[1],
                "投放点z": release_position2[2],
                "起爆点x": explosion_position2[0],
                "起爆点y": explosion_position2[1],
                "起爆点z": explosion_position2[2],
                "有效干扰时长": duration2,
                "投放时间": release_time2,  # 添加投放时间
```

```
            "引爆时间": delay2,  # 添加引爆时间
        }
    )

    # 计算FY3数据
    direction3_x = np.cos(azimuth3)
    direction3_y = np.sin(azimuth3)
    direction3_z = 0
    direction3 = (direction3_x, direction3_y, direction3_z)

    direction3_angle = (np.arctan2(direction3_y, direction3_x) *
       180 / np.pi) % 360

    dir3_norm = np.sqrt(direction3_x**2 + direction3_y**2)
    plane3_target = (
        FY3_initial_pos[0] + direction3_x / dir3_norm *
           target_distance,
        FY3_initial_pos[1] + direction3_y / dir3_norm *
           target_distance,
        FY3_initial_pos[2],
    )

    release_position3 = calculate_plane_position(
        FY3_initial_pos, direction3, speed3, release_time3
    )

    explosion_position3 = calculate_bomb_position(
        FY3_initial_pos, direction3, speed3, release_time3,
           delay3
    )

    duration3, _ = calculate_shielding_duration(
        missile_initial_pos=M1_initial_pos,
        missile_velocity=300,
        missile_target=(0, 0, 0),
        plane_initial_pos=FY3_initial_pos,
        plane_target=plane3_target,
        plane_speed=speed3,
        release_time=release_time3,
        explosion_delay=delay3,
```

```
            sample_points=500,
            verbose=False,
        )

        drones_data.append(
            {
                "编号": "FY3",
                "运动方向": direction3_angle,
                "运动速度": speed3,
                "投放点x": release_position3[0],
                "投放点y": release_position3[1],
                "投放点z": release_position3[2],
                "起爆点x": explosion_position3[0],
                "起爆点y": explosion_position3[1],
                "起爆点z": explosion_position3[2],
                "有效干扰时长": duration3,
                "投放时间": release_time3,  # 添加投放时间
                "引爆时间": delay3,  # 添加引爆时间
            }
        )

        # 计算总有效干扰时长（高精度评估）
        total_shielding_periods = []

        # 收集三架无人机的所有遮蔽区间
        for drone_idx, (pos, dir, speed, release_time, delay) in enumerate(
            [
                (FY1_initial_pos, direction1, speed1, release_time1, delay1),
                (FY2_initial_pos, direction2, speed2, release_time2, delay2),
                (FY3_initial_pos, direction3, speed3, release_time3, delay3),
            ]
        ):
            # 计算目标位置
            dir_x, dir_y, dir_z = dir
            dir_norm = np.sqrt(dir_x**2 + dir_y**2)
            target_distance = 20000
```

```
            plane_target = (
                pos[0] + dir_x / dir_norm * target_distance,
                pos[1] + dir_y / dir_norm * target_distance,
                pos[2],
            )

            # 计算遮蔽区间
            try:
                _, shield_times = calculate_shielding_duration(
                    missile_initial_pos=M1_initial_pos,
                    missile_velocity=300,
                    missile_target=(0, 0, 0),
                    plane_initial_pos=pos,
                    plane_target=plane_target,
                    plane_speed=speed,
                    release_time=release_time,
                    explosion_delay=delay,
                    sample_points=500,
                    verbose=False,
                )
                total_shielding_periods.extend(shield_times)
            except Exception as e:
                print(f"计算无人机{drone_idx+1}的遮蔽区间时出错: {e}")

        # 合并重叠区间
        if total_shielding_periods:
            total_shielding_periods.sort(key=lambda x: x[0])
            merged_periods = [total_shielding_periods[0]]

            for current in total_shielding_periods[1:]:
                previous = merged_periods[-1]
                if current[0] <= previous[1]:
                    merged_periods[-1] = (previous[0], max(previous[1], current[1]))
                else:
                    merged_periods.append(current)

            total_duration = sum(end - start for start, end in merged_periods)
```

```
        else:
            total_duration = 0.0
            merged_periods = []

        # 创建结果DataFrame
        df = pd.DataFrame(
            columns=[
                "无人机编号",
                "无人机运动方向",
                "无人机运动速度 (m/s)",
                "烟幕干扰弹投放点的x坐标 (m)",
                "烟幕干扰弹投放点的y坐标 (m)",
                "烟幕干扰弹投放点的z坐标 (m)",
                "烟幕干扰弹起爆点的x坐标 (m)",
                "烟幕干扰弹起爆点的y坐标 (m)",
                "烟幕干扰弹起爆点的z坐标 (m)",
                "有效干扰时长 (s)",
            ]
        )

        # 填充数据
        for i, drone in enumerate(drones_data):
            df.loc[i] = [
                drone["编号"],
                drone["运动方向"],
                drone["运动速度"],
                drone["投放点x"],
                drone["投放点y"],
                drone["投放点z"],
                drone["起爆点x"],
                drone["起爆点y"],
                drone["起爆点z"],
                drone["有效干扰时长"],
            ]

        # 输出优化结果
        print("\n=== 三架无人机协同投放PSO优化结果 ===")
        print(f"总有效干扰时长: {total_duration:.4f}秒")
        for i, drone in enumerate(drones_data):
            drone_id = drone["编号"]
```

```python
            print(f"\n{drone_id}:")
            print(f"  运动方向: {drone['运动方向']:.2f}度")
            print(f"  运动速度: {drone['运动速度']:.2f} m/s")
            print(
                f"  投放点: ({drone['投放点x']:.2f}, {drone['投放点y']:.2f}, {drone['投放点z']:.2f})"
            )
            print(
                f"  起爆点: ({drone['起爆点x']:.2f}, {drone['起爆点y']:.2f}, {drone['起爆点z']:.2f})"
            )
            print(f"  有效干扰时长: {drone['有效干扰时长']:.2f}秒")
            print(f"  投放时间: {drone['投放时间']:.2f}秒")  # 输出投放时间
            print(f"  引爆时间: {drone['引爆时间']:.2f}秒")  # 输出引爆时间

        # 分析遮蔽时间段
        print("\n=== 遮蔽时间段分析 ===")
        print(f"共有{len(merged_periods)}个遮蔽时间段:")
        for i, (start, end) in enumerate(merged_periods):
            print(f"段{i+1}: {start:.2f}s - {end:.2f}s (持续{end-start:.2f}s)")

        end_time = time.time()
        print(f"\nPSO优化总耗时: {end_time - start_time:.2f}秒")

        return global_best_position, drones_data, total_duration, merged_periods

    finally:
        # 关闭进程池
        pool.close()
        pool.join()


def run_pso_optimization():
    """运行PSO优化"""

    print("开始粒子群优化三架无人机协同投放烟幕弹...")
```

```python

    try:
        # 运行优化
        best_position, drones_data, total_duration, shield_periods = optimize_with_pso()
        print(f"\n优化成功完成! 最大总遮蔽时长: {total_duration:.4f}秒")

    except Exception as e:
        print(f"优化过程中出错: {e}")
        import traceback

        traceback.print_exc()


if __name__ == "__main__":
    run_pso_optimization()




import numpy as np
import random
import matplotlib.pyplot as plt
import pandas as pd
import time
import multiprocessing
from functools import lru_cache


def is_effectively_shielded(
    missile_pos,
    target_points,
    cloud_center,
    explosion_time,
    current_time,
    smoke_radius=10,
    last_time=20,
```

```
):
    """
    判断导弹是否被有效遮蔽

    参数:
    missile_pos -- 导弹当前位置坐标 (x,y,z)
    target_points -- 目标上的采样点列表
    cloud_center -- 烟幕云团中心位置 (x,y,z)
    explosion_time -- 烟幕弹爆炸时间
    current_time -- 当前时间
    smoke_radius -- 烟幕云团半径,10m
    last_time -- 烟幕有效持续时间,20s

    返回:
    bool -- 是否被有效遮蔽
    """
    # 时间检查
    if current_time - explosion_time > last_time:
        return False

    # 位置检查
    for target_point in target_points:
        # 导弹到目标点的视线向量
        sight_vector = np.array(target_point) - np.array(missile_pos)
        sight_length = np.linalg.norm(sight_vector)
        if sight_length < 1e-6:  # 避免除零
            return True

        # 云团中心到视线的投影比例
        mc_vector = np.array(cloud_center) - np.array(missile_pos)
        t = np.dot(mc_vector, sight_vector) / (sight_length**2)

        # 云团中心到视线的垂直距离
        distance = np.linalg.norm(mc_vector - t * sight_vector)

        # 云团是否在导弹和目标之间
        in_between = 0 <= t <= 1

        # 如果视线未被遮蔽或云团不在中间，则整体未被遮蔽
        if distance >= smoke_radius:
```

```
            # 如果云团离视线太远，肯定不遮蔽
            return False
        elif not in_between:
            # 如果云团不在导弹和目标之间，检查是否足够接近目标
            if t > 1:  # 云团在目标"后方"
                distance_to_target = np.linalg.norm(
                    np.array(cloud_center) - np.array(target_point)
                )
                return distance_to_target < smoke_radius
            else:  # 云团在导弹"后方"(t<0)
                distance_to_missile = np.linalg.norm(
                    np.array(cloud_center) - np.array(missile_pos)
                )
                return distance_to_missile < smoke_radius
    return True


def is_effectively_shielded_by_multiple_clouds(
    missile_pos,
    target_points,
    cloud_centers,
    explosion_times,
    current_time,
    smoke_radius=10,
    last_time=20,
):
    """
    判断导弹是否被多个烟幕云团有效遮蔽

    参数:
    missile_pos -- 导弹当前位置坐标 (x,y,z)
    target_points -- 目标上的采样点列表
    cloud_centers -- 多个烟幕云团中心位置列表，每个元素为 (x,y,z) 或 None
    explosion_times -- 多个烟幕弹爆炸时间列表
    current_time -- 当前时间
    smoke_radius -- 烟幕云团半径,10m
    last_time -- 烟幕有效持续时间,20s

    返回:
```

```
    bool -- 是否被有效遮蔽
    """
    # 筛选有效的云团(已爆炸且未超过持续时间)
    active_clouds = []
    for cloud_center, explosion_time in zip(cloud_centers, explosion_times):
        if cloud_center is not None and current_time - explosion_time <= last_time:
            active_clouds.append(cloud_center)

    # 如果没有有效云团, 直接返回False
    if not active_clouds:
        return False

    # 检查每个目标点
    for target_point in target_points:
        # 导弹到目标点的视线向量
        sight_vector = np.array(target_point) - np.array(missile_pos)
        sight_length = np.linalg.norm(sight_vector)

        # 如果导弹已经到达目标点, 视为被遮蔽
        if sight_length < 1e-6:
            return True

        # 检查是否至少有一个云团遮蔽了视线
        line_shielded = False

        for cloud_center in active_clouds:
            # 云团中心到视线的投影比例
            mc_vector = np.array(cloud_center) - np.array(missile_pos)
            t = np.dot(mc_vector, sight_vector) / (sight_length**2)

            # 云团中心到视线的垂直距离
            distance = np.linalg.norm(mc_vector - t * sight_vector)

            # 云团是否在导弹和目标之间
            in_between = 0 <= t <= 1

            # 判断该云团是否遮蔽了视线
```

```
            if distance < smoke_radius and in_between:
                line_shielded = True
                break
            elif distance < smoke_radius:
                # 如果云团不在导弹和目标之间，检查是否足够接近目标或导弹
                if t > 1:  # 云团在目标"后方"
                    distance_to_target = np.linalg.norm(
                        np.array(cloud_center) - np.array(target_point)
                    )
                    if distance_to_target < smoke_radius:
                        line_shielded = True
                        break
                else:  # 云团在导弹"后方"(t<0)
                    distance_to_missile = np.linalg.norm(
                        np.array(cloud_center) - np.array(missile_pos)
                    )
                    if distance_to_missile < smoke_radius:
                        line_shielded = True
                        break

        # 如果所有云团都未能遮蔽该视线，则整体未被遮蔽
        if not line_shielded:
            return False

    # 所有视线都被遮蔽，则整体被遮蔽
    return True


def calculate_missile_position(initial_pos, time, velocity=300, target_pos=(0, 0, 0)):
    """
    计算导弹在给定时间的位置

    参数:
    initial_pos -- 导弹初始位置 (x,y,z)
    time -- 时间(秒)
    velocity -- 导弹速度(米/秒),默认300
```

```
    target_pos -- 导弹目标位置(假目标), 默认(0,0,0)

    返回:
    tuple -- 导弹当前位置 (x,y,z)
    """
    initial_pos = np.array(initial_pos)
    target_pos = np.array(target_pos)

    direction = target_pos - initial_pos
    direction_unit = direction / np.linalg.norm(direction)
    velocity_vector = direction_unit * velocity

    new_pos = initial_pos + velocity_vector * time
    return tuple(new_pos)


def calculate_plane_position(initial_pos, direction, speed, time):
    """
    计算飞机在给定时间的位置

    参数:
    initial_pos -- 飞机初始位置 (x,y,z)
    direction -- 飞行方向向量
    speed -- 飞行速度(米/秒)
    time -- 时间(秒)

    返回:
    tuple -- 飞机当前位置 (x,y,z)
    """
    initial_pos = np.array(initial_pos)
    direction = np.array(direction)

    direction_unit = direction / np.linalg.norm(direction)
    velocity_vector = direction_unit * speed

    new_pos = initial_pos + velocity_vector * time
    return tuple(new_pos)


def calculate_bomb_position(
```

```
    plane_initial_pos,
    plane_direction,
    plane_speed,
    release_time,
    fall_time,
    gravity=9.8,
):
    """
    计算烟幕弹轨迹和爆炸位置

    参数:
    plane_initial_pos -- 无人机初始位置 (x,y,z)
    plane_direction -- 无人机飞行方向
    plane_speed -- 无人机速度(米/秒)
    release_time -- 投放时间(秒)
    fall_time -- 烟幕弹下落时间(秒)
    gravity -- 重力加速度,默认9.8

    返回:
    tuple -- 爆炸位置 (x,y,z)
    """
    # 计算投放位置
    release_position = calculate_plane_position(
        plane_initial_pos, plane_direction, plane_speed, release_time
    )

    # 计算水平位移
    plane_direction = np.array(plane_direction)
    direction_unit = plane_direction / np.linalg.norm(plane_direction
        )
    velocity_vector = direction_unit * plane_speed

    # 计算爆炸位置
    explosion_pos = (
        release_position[0] + velocity_vector[0] * fall_time,
        release_position[1] + velocity_vector[1] * fall_time,
        release_position[2] - 0.5 * gravity * fall_time**2,
    )

    return explosion_pos
```

```python


def calculate_cloud_position(t, explosion_position, explosion_time, sink_speed=3):
    """
    计算特定时刻的云团位置

    参数:
    t -- 当前时间(秒)
    explosion_position -- 爆炸位置 (x,y,z)
    explosion_time -- 爆炸时间(秒)
    sink_speed -- 云团下沉速度(米/秒),默认3

    返回:
    list/None -- 云团位置 [x,y,z] 或 None(未爆炸)
    """
    # 云团匀速下沉
    if t < explosion_time:
        return None  # 未起爆
    else:
        return [
            explosion_position[0],
            explosion_position[1],
            explosion_position[2] - sink_speed * (t - explosion_time),
        ]


def generate_cylinder_sample_points(
    base_center=(0, 200, 0), height=10, radius=7, num_points=5000
):
    """
    生成圆柱体目标上的采样点

    参数:
    base_center -- 底面圆心坐标, 默认(0,200,0)
    height -- 圆柱体高度, 默认10
    radius -- 圆柱体半径, 默认7
    num_points -- 每个圆面上的采样点数, 默认100

```

```
    返回:
    list -- 圆柱体表面采样点列表
    """
    base_center = np.array(base_center)
    top_center = np.array([base_center[0], base_center[1],
        base_center[2] + height])

    points = []

    # 底面采样点
    for i in range(num_points):
        angle = 2 * np.pi * i / num_points
        x = base_center[0] + radius * np.cos(angle)
        y = base_center[1] + radius * np.sin(angle)
        z = base_center[2]
        points.append((x, y, z))

    # 顶面采样点
    for i in range(num_points):
        angle = 2 * np.pi * i / num_points
        x = top_center[0] + radius * np.cos(angle)
        y = top_center[1] + radius * np.sin(angle)
        z = top_center[2]
        points.append((x, y, z))

    return points


def calculate_shielding_duration(
    missile_initial_pos=(20000, 0, 2000),
    missile_velocity=300,
    missile_target=(0, 0, 0),
    plane_initial_pos=(17800, 0, 1800),
    plane_target=(0, 0, 1800),
    plane_speed=120,
    release_time=1.5,
    explosion_delay=3.6,
    smoke_radius=10,
    smoke_last_time=20,
    cloud_sink_speed=3,
```

```
    target_center=(0, 200, 0),
    target_height=10,
    target_radius=7,
    sample_points=5000,
    time_step=0.005,
    verbose=True,
):
    """
    计算导弹被烟幕有效遮蔽的总时长

    参数:
    missile_initial_pos -- 导弹初始位置, 默认(20000,0,2000)
    missile_velocity -- 导弹速度(米/秒), 默认300
    missile_target -- 导弹目标位置(假目标), 默认(0,0,0)
    plane_initial_pos -- 无人机初始位置, 默认(17800,0,1800)
    plane_target -- 无人机目标位置, 默认(0,0,0)
    plane_speed -- 无人机速度(米/秒), 默认120
    release_time -- 烟幕弹投放时间(秒), 默认1.5
    explosion_delay -- 烟幕弹起爆延迟(秒), 默认3.6
    smoke_radius -- 烟幕云团半径(米), 默认10
    smoke_last_time -- 烟幕有效持续时间(秒), 默认20
    cloud_sink_speed -- 云团下沉速度(米/秒), 默认3
    target_center -- 真目标底面圆心坐标, 默认(0,200,0)
    target_height -- 真目标高度(米), 默认10
    target_radius -- 真目标半径(米), 默认7
    sample_points -- 圆柱体每个面的采样点数, 默认100
    time_step -- 时间步长(秒), 默认0.01
    verbose -- 是否输出详细信息, 默认True

    返回:
    tuple -- (总遮蔽时长(秒), 遮蔽时间段列表[(开始时间,结束时间),...]
    """
    # 计算无人机飞行方向
    plane_direction = np.array(plane_target) - np.array(
        plane_initial_pos)

    # 计算爆炸位置和时间
    explosion_position = calculate_bomb_position(
        plane_initial_pos, plane_direction, plane_speed, release_time
            , explosion_delay
```

```
    )

    explosion_time = release_time + explosion_delay

    # 生成目标采样点
    target_points = generate_cylinder_sample_points(
        target_center, target_height, target_radius, sample_points
    )

    if verbose:
        print("=== 烟幕干扰分析 ===")
        print(f"无人机初始位置: {plane_initial_pos}")
        print(f"飞行速度: {plane_speed} m/s, 朝向目标: {plane_target}")
        print(f"投放时间: {release_time}s")
        print(f"起爆延时: {explosion_delay}s")
        print(f"爆炸时间: {explosion_time}s")
        print(
            f"爆炸位置: ({explosion_position[0]:.1f}, {explosion_position[1]:.1f}, {explosion_position[2]:.1f})"
        )
        print(f"烟幕半径: {smoke_radius}m")
        print(f"烟幕有效时间: {smoke_last_time}s")
        print(f"云团下沉速度: {cloud_sink_speed} m/s")
        print(f"\n目标信息:")
        print(
            f"真目标位置: 圆心{target_center}, 半径{target_radius}m, 高{target_height}m"
        )
        print(f"采样点数: {len(target_points)}")
        print(f"\n开始模拟...")

    # 遍历时间步长
    time_start = 0
    time_end = 100  # 足够长的时间范围

    shielded_times = []
    is_currently_shielded = False
    shield_start_time = None
```

```python

    # 记录关键时刻信息
    first_shield_info = None
    last_shield_info = None

    for t in np.arange(time_start, time_end, time_step):
        # 计算当前位置
        missile_pos = calculate_missile_position(
            missile_initial_pos, t, missile_velocity, missile_target
        )
        cloud_pos = calculate_cloud_position(
            t, explosion_position, explosion_time, cloud_sink_speed
        )

        # 检查导弹是否已经通过目标区域
        missile_pos_array = np.array(missile_pos)
        if np.linalg.norm(missile_pos_array) < 1:
            if is_currently_shielded:
                shielded_times.append((shield_start_time, t))
                last_shield_info = {
                    "time": t,
                    "missile_pos": missile_pos,
                    "cloud_pos": cloud_pos,
                    "missile_distance": np.linalg.norm(
                        missile_pos_array - np.array(target_center)
                    ),
                }
            break

        # 检查云团是否已形成
        if cloud_pos is None:
            continue

        # 判断是否有效遮蔽
        is_shielded = is_effectively_shielded(
            missile_pos,
            target_points,
            cloud_pos,
            explosion_time,
            t,
```

```
            smoke_radius,
            smoke_last_time,
        )

        # 记录遮蔽时间段
        if is_shielded and not is_currently_shielded:
            is_currently_shielded = True
            shield_start_time = t
            if first_shield_info is None:
                first_shield_info = {
                    "time": t,
                    "missile_pos": missile_pos,
                    "cloud_pos": cloud_pos,
                    "missile_distance": np.linalg.norm(
                        np.array(missile_pos) - np.array(
                            target_center)
                    ),
                }
        elif not is_shielded and is_currently_shielded:
            is_currently_shielded = False
            shielded_times.append((shield_start_time, t))
            last_shield_info = {
                "time": t,
                "missile_pos": missile_pos,
                "cloud_pos": cloud_pos,
                "missile_distance": np.linalg.norm(
                    np.array(missile_pos) - np.array(target_center)
                ),
            }

    # 输出详细信息
    if verbose:
        print(f"\n=== 遮蔽时间段详情 ===")
        if len(shielded_times) == 0:
            print("未检测到有效遮蔽！")
            # 调试信息
            t_test = 10.0  # 测试时刻
            missile_test = calculate_missile_position(
                missile_initial_pos, t_test, missile_velocity,
                    missile_target
```

```
            )
            cloud_test = calculate_cloud_position(
                t_test, explosion_position, explosion_time, cloud_sink_speed
            )
            if cloud_test:
                print(f"\n调试信息:")
                print(f"t={t_test}s时:")
                print(f"  导弹位置: {missile_test}")
                print(f"  云团位置: {cloud_test}")
                print(
                    f"  导弹到真目标距离: {np.linalg.norm(np.array(missile_test) - np.array(target_center)):.1f}m"
                )
        else:
            for i, (start, end) in enumerate(shielded_times):
                print(f"第{i+1}段: {start:.2f}s - {end:.2f}s (持续 {end-start:.2f}s)")

            if first_shield_info:
                print(f"\n首次遮蔽时刻 (t={first_shield_info['time']:.2f}s):")
                print(f"  导弹位置: {first_shield_info['missile_pos']}")
                print(f"  云团位置: {first_shield_info['cloud_pos']}")
                print(f"  导弹距真目标: {first_shield_info['missile_distance']:.1f}m")

            if last_shield_info:
                print(f"\n最后遮蔽时刻 (t={last_shield_info['time']:.2f}s):")
                print(f"  导弹位置: {last_shield_info['missile_pos']}")
                print(f"  云团位置: {last_shield_info['cloud_pos']}")
                print(f"  导弹距真目标: {last_shield_info['missile_distance']:.1f}m")

        # 计算总遮蔽时长
```

```
        total_duration = sum(end - start for start, end in shielded_times)
        print(f"\n=== 结果汇总 ===")
        print(f"遮蔽段数: {len(shielded_times)}")
        print(f"总遮蔽时长: {total_duration:.4f} 秒")

    return sum(end - start for start, end in shielded_times), shielded_times


def is_effectively_shielded_by_multiple_clouds(
    missile_pos,
    target_points,
    cloud_centers,
    explosion_times,
    current_time,
    smoke_radius=10,
    last_time=20,
):
    """
    判断导弹是否被多个烟幕云团有效遮蔽

    参数:
    missile_pos -- 导弹当前位置坐标 (x,y,z)
    target_points -- 目标上的采样点列表
    cloud_centers -- 多个烟幕云团中心位置列表, 每个元素为 (x,y,z) 或 None
    explosion_times -- 多个烟幕弹爆炸时间列表
    current_time -- 当前时间
    smoke_radius -- 烟幕云团半径,10m
    last_time -- 烟幕有效持续时间,20s

    返回:
    bool -- 是否被有效遮蔽
    """
    # 筛选有效的云团(已爆炸且未超过持续时间)
    active_clouds = []
    for cloud_center, explosion_time in zip(cloud_centers, explosion_times):
        if cloud_center is not None and current_time - explosion_time
```

```
            <= last_time:
            active_clouds.append(cloud_center)

    # 如果没有有效云团，直接返回False
    if not active_clouds:
        return False

    # 检查每个目标点
    for target_point in target_points:
        # 导弹到目标点的视线向量
        sight_vector = np.array(target_point) - np.array(missile_pos)
        sight_length = np.linalg.norm(sight_vector)

        # 如果导弹已经到达目标点，视为被遮蔽
        if sight_length < 1e-6:
            return True

        # 检查是否至少有一个云团遮蔽了视线
        line_shielded = False

        for cloud_center in active_clouds:
            # 云团中心到视线的投影比例
            mc_vector = np.array(cloud_center) - np.array(missile_pos
                )
            t = np.dot(mc_vector, sight_vector) / (sight_length**2)

            # 云团中心到视线的垂直距离
            distance = np.linalg.norm(mc_vector - t * sight_vector)

            # 云团是否在导弹和目标之间
            in_between = 0 <= t <= 1

            # 判断该云团是否遮蔽了视线
            if distance < smoke_radius and in_between:
                line_shielded = True
                break
            elif distance < smoke_radius:
                # 如果云团不在导弹和目标之间，检查是否足够接近目标或
                    导弹
                if t > 1:  # 云团在目标"后方"
```

```
                    distance_to_target = np.linalg.norm(
                        np.array(cloud_center) - np.array(
                            target_point)
                    )
                    if distance_to_target < smoke_radius:
                        line_shielded = True
                        break
                else:  # 云团在导弹"后方"(t<0)
                    distance_to_missile = np.linalg.norm(
                        np.array(cloud_center) - np.array(missile_pos
                            )
                    )
                    if distance_to_missile < smoke_radius:
                        line_shielded = True
                        break

        # 如果所有云团都未能遮蔽该视线，则整体未被遮蔽
        if not line_shielded:
            return False

    # 所有视线都被遮蔽，则整体被遮蔽
    return True


def calculate_multi_smoke_shielding_duration(
    missile_initial_pos,
    missile_velocity,
    missile_target,
    plane_initial_positions,
    plane_targets,
    plane_speeds,
    release_times,
    explosion_delays,
    target_center=(0, 200, 0),
    target_height=10,
    target_radius=7,
    sample_points=100,
    time_step=0.05,
    verbose=False,
):
```

```python
    """计算多枚烟幕弹对单枚导弹的协同遮蔽时长"""
    # 计算所有无人机的飞行方向
    plane_directions = []
    for plane_initial_pos, plane_target in zip(
        plane_initial_positions, plane_targets):
        direction = np.array(plane_target) - np.array(
            plane_initial_pos)
        direction_norm = np.linalg.norm(direction)
        if direction_norm > 0:
            direction = direction / direction_norm
        else:
            direction = np.array([1, 0, 0])  # 默认方向
        plane_directions.append(tuple(direction))

    # 计算每枚烟幕弹的爆炸位置和时间
    explosion_positions = []
    explosion_times = []

    for (
        plane_initial_pos,
        plane_direction,
        plane_speed,
        release_time,
        explosion_delay,
    ) in zip(
        plane_initial_positions,
        plane_directions,
        plane_speeds,
        release_times,
        explosion_delays,
    ):
        explosion_pos = calculate_bomb_position(
            plane_initial_pos,
            plane_direction,
            plane_speed,
            release_time,
            explosion_delay,
        )
        explosion_time = release_time + explosion_delay

```

```
        explosion_positions.append(explosion_pos)
        explosion_times.append(explosion_time)

    # 生成目标采样点
    target_points = generate_cylinder_sample_points(
        target_center, target_height, target_radius, sample_points
    )

    # 遍历时间步长
    time_start = 0
    time_end = 100  # 足够长的时间范围

    shielded_times = []
    is_currently_shielded = False
    shield_start_time = None

    for t in np.arange(time_start, time_end, time_step):
        # 计算当前导弹位置
        missile_pos = calculate_missile_position(
            missile_initial_pos, t, missile_velocity, missile_target
        )

        # 检查导弹是否已经通过目标区域
        missile_pos_array = np.array(missile_pos)
        if np.linalg.norm(missile_pos_array) < 1:
            if is_currently_shielded:
                shielded_times.append((shield_start_time, t))
            break

        # 计算当前各云团位置
        cloud_positions = []
        for explosion_pos, explosion_time in zip(explosion_positions,
            explosion_times):
            cloud_pos = calculate_cloud_position(
                t, explosion_pos, explosion_time, 3  #
                    cloud_sink_speed=3
            )
            cloud_positions.append(cloud_pos)

        # 判断是否有效遮蔽
```

```
        is_shielded = is_effectively_shielded_by_multiple_clouds(
            missile_pos,
            target_points,
            cloud_positions,
            explosion_times,
            t,
            10,  # smoke_radius=10
            20,  # smoke_last_time=20
        )

        # 记录遮蔽时间段
        if is_shielded and not is_currently_shielded:
            is_currently_shielded = True
            shield_start_time = t
        elif not is_shielded and is_currently_shielded:
            is_currently_shielded = False
            shielded_times.append((shield_start_time, t))

    # 计算总遮蔽时长
    total_duration = sum(end - start for start, end in shielded_times
        )
    return total_duration, shielded_times


def enforce_release_time_constraints(position):
    """
    对单个粒子的位置向量强制施加投放时间间隔至少为1秒的约束。
    该函数会按时间顺序重新排列烟幕弹，并确保它们之间有足够的时间间
        隔。
    """
    for i in range(5):  # 遍历5架无人机
        base_idx = i * 8

        # 提取每枚烟幕弹的（投放时间，延迟时间）对
        releases = []
        for j in range(3):  # 3枚烟幕弹
            time_idx = base_idx + 2 + j * 2
            delay_idx = base_idx + 3 + j * 2
            releases.append([position[time_idx], position[delay_idx
                ]])
```

```python

        # 1. 关键一步：按投放时间对烟幕弹进行排序
        #    这确保了我们总是比较时间上相邻的两次投放。
        releases.sort(key=lambda x: x[0])

        # 2. 强制施加1秒的时间间隔
        #    比较第2枚和第1枚
        if releases[1][0] < releases[0][0] + 1.0:
            releases[1][0] = releases[0][0] + 1.0
        #    比较第3枚和第2枚
        if releases[2][0] < releases[1][0] + 1.0:
            releases[2][0] = releases[1][0] + 1.0

        # 3. 将修复后的值写回原始位置向量
        for j in range(3):
            time_idx = base_idx + 2 + j * 2
            delay_idx = base_idx + 3 + j * 2
            position[time_idx] = releases[j][0]
            position[delay_idx] = releases[j][1]

    return position


drone_positions = [
    (17800, 0, 1800),  # FY1
    (12000, 1400, 1400),  # FY2
    (6000, -3000, 700),  # FY3
    (15000, -2000, 1200),  # FY4
    (9000, 2500, 900),  # FY5
]

missile_positions = [
    (20000, 0, 2000),  # M1
    (19000, 2000, 2200),  # M2
    (18000, -1500, 1900),  # M3
]

# 真目标位置
target_center = (0, 200, 0)

```

```python

def calculate_total_shielding_duration(
    drone_params,
    drone_positions,
    missile_positions,
    target_center=(0, 200, 0),
    sample_points=50,
):
    """计算总的遮蔽时长(所有导弹的总和)"""
    total_duration = 0.0

    # 对每枚导弹计算遮蔽效果
    for missile_idx, missile_pos in enumerate(missile_positions):
        # 收集所有无人机针对该导弹的所有烟幕弹参数
        all_bomb_params = []

        for drone_idx, drone_param in enumerate(drone_params):
            # 计算无人机的飞行方向向量
            azimuth = drone_param["azimuth"]
            speed = drone_param["speed"]

            direction_x = np.cos(azimuth)
            direction_y = np.sin(azimuth)
            direction = (direction_x, direction_y, 0)

            # 计算目标位置
            dir_norm = np.sqrt(direction_x**2 + direction_y**2)
            if dir_norm < 1e-6:  # 避免除零
                continue

            target_distance = 20000
            drone_target = (
                drone_positions[drone_idx][0]
                + direction_x / dir_norm * target_distance,
                drone_positions[drone_idx][1]
                + direction_y / dir_norm * target_distance,
                drone_positions[drone_idx][2],
            )

            # 添加该无人机的所有烟幕弹参数
```

```
            for release_time, explosion_delay in drone_param["releases"]:
                if release_time <= 20:  # 只考虑实际投放的烟幕弹（不超过20秒）
                    all_bomb_params.append(
                        {
                            "plane_initial_pos": drone_positions[drone_idx],
                            "plane_target": drone_target,
                            "plane_speed": speed,
                            "release_time": release_time,
                            "explosion_delay": explosion_delay,
                        }
                    )

        # 如果有针对该导弹的烟幕弹，计算遮蔽时长
        if all_bomb_params:
            # 限制每枚导弹最多考虑10枚烟幕弹（性能优化）
            all_bomb_params.sort(key=lambda x: x["release_time"])  # 按投放时间排序
            bomb_count = min(len(all_bomb_params), 10)
            selected_bombs = all_bomb_params[:bomb_count]

            # 提取参数
            plane_initial_positions = [
                bomb["plane_initial_pos"] for bomb in selected_bombs
            ]
            plane_targets = [bomb["plane_target"] for bomb in selected_bombs]
            plane_speeds = [bomb["plane_speed"] for bomb in selected_bombs]
            release_times = [bomb["release_time"] for bomb in selected_bombs]
            explosion_delays = [bomb["explosion_delay"] for bomb in selected_bombs]

            # 计算针对该导弹的遮蔽时长
            try:
                duration, _ = calculate_multi_smoke_shielding_duration(
```

```
                    missile_initial_pos=missile_pos,
                    missile_velocity=300,
                    missile_target=(0, 0, 0),
                    plane_initial_positions=plane_initial_positions,
                    plane_targets=plane_targets,
                    plane_speeds=plane_speeds,
                    release_times=release_times,
                    explosion_delays=explosion_delays,
                    target_center=target_center,
                    sample_points=sample_points,
                    time_step=0.05,  # 增大时间步长以加速计算
                    verbose=False,
                )
                total_duration += duration
            except Exception as e:
                # 如果计算出错，忽略该导弹
                pass

    # 添加奖励：鼓励平均分配遮蔽效果和资源利用
    if total_duration > 0:
        # 计算无人机参与度 - 每架无人机投放的烟幕弹数量
        drone_participation = [0] * 5
        for i in range(5):
            for release_time, _ in drone_params[i]["releases"]:
                if release_time <= 20:  # 考虑实际投放的烟幕弹
                    drone_participation[i] += 1

        # 奖励多架无人机参与
        active_drones = sum(1 for p in drone_participation if p > 0)
        if active_drones >= 3:
            total_duration *= 1.1  # 奖励至少3架无人机参与
        if active_drones >= 4:
            total_duration *= 1.1  # 更多奖励4架及以上参与

        # 奖励平均分布的投放时间
        all_release_times = []
        for drone in drone_params:
            for release_time, _ in drone["releases"]:
                if release_time <= 20:
                    all_release_times.append(release_time)
```

```python

        if all_release_times:
            time_range = max(all_release_times) - min(
                all_release_times)
            if time_range > 10:  # 如果投放时间跨度大，给予奖励
                total_duration *= 1.05

    return total_duration


fitness_cache = {}


def position_to_key(position):
    """将位置向量转换为可哈希的键，用于缓存"""
    # 截断到小数点后4位以增加缓存命中率
    return tuple(float(f"{x:.4f}") for x in position)


def evaluate_position(position):
    """评估一个位置(解)的适应度"""
    # 检查缓存
    key = position_to_key(position)
    if key in fitness_cache:
        return fitness_cache[key]

    # 解码参数
    drone_params = []
    for i in range(5):  # 5架无人机
        drone_param = {
            "azimuth": position[i * 8],
            "speed": position[i * 8 + 1],
            "releases": [],
        }

        # 添加3枚烟幕弹的参数
        for j in range(3):
            release_time = position[i * 8 + 2 + j * 2]
            explosion_delay = position[i * 8 + 3 + j * 2]
            drone_param["releases"].append((release_time,
```

```
                explosion_delay))

        drone_params.append(drone_param)

    # 计算对3枚导弹的总遮蔽时长
    total_duration = calculate_total_shielding_duration(
        drone_params, drone_positions, missile_positions,
            target_center
    )

    # 缓存结果
    fitness_cache[key] = total_duration
    return total_duration


def optimize_with_pso_five_vs_three():
    """使用粒子群算法优化5架无人机对抗3枚导弹的策略"""
    start_time = time.time()

    # 定义5架无人机和3枚导弹的初始位置

    # 定义参数边界 - 每架无人机的飞行方向、速度、每枚烟幕弹的投放时间
        和延迟
    # 每架无人机8个参数: 方向角、速度、3枚烟幕弹的投放时间和延迟
    bounds = []
    for _ in range(5):  # 5架无人机
        # 方向角范围 [-pi/2, pi/2]
        bounds.append((-np.pi, np.pi))
        # 速度范围 [80, 140] m/s
        bounds.append((80, 140))

        # 3枚烟幕弹的参数
        for _ in range(3):
            # 投放时间范围 [0, 40] 秒
            bounds.append((0, 40))
            # 延迟时间范围 [0, 5] 秒
            bounds.append((0, 6))

    # 总共40个参数: 5架无人机 × (1个方向角 + 1个速度 + 3枚烟幕弹×2个
        参数)
```

```python

    # 创建并行管理器
    manager = multiprocessing.Manager()
    fitness_cache = manager.dict()

    class Particle:
        """粒子类，代表PSO中的一个候选解"""

        def __init__(self, dimensions):
            """初始化一个粒子"""
            # 随机生成位置向量
            self.position = np.array(
                [random.uniform(low, high) for low, high in bounds]
            )
            self.position = enforce_release_time_constraints(self.position)

            # 随机生成速度向量
            self.velocity = np.array(
                [
                    random.uniform(-0.3 * (high - low), 0.3 * (high - low))
                    for low, high in bounds
                ]
            )

            # 初始化最佳位置和适应度
            self.best_position = self.position.copy()
            self.best_fitness = -float("inf")
            self.fitness = -float("inf")

        def update_velocity(self, global_best_position, w=0.7, c1=1.5, c2=1.5):
            """更新粒子的速度"""
            r1 = np.random.rand(len(self.position))
            r2 = np.random.rand(len(self.position))

            cognitive_component = c1 * r1 * (self.best_position - self.position)
            social_component = c2 * r2 * (global_best_position - self
```

```
                .position)

            self.velocity = w * self.velocity + cognitive_component +
                social_component

        def update_position(self):
            """更新粒子的位置并确保在边界内"""
            self.position = self.position + self.velocity

            # 边界处理
            for i in range(len(self.position)):
                if self.position[i] < bounds[i][0]:
                    self.position[i] = bounds[i][0]
                    self.velocity[i] *= -0.5  # 速度反弹
                elif self.position[i] > bounds[i][1]:
                    self.position[i] = bounds[i][1]
                    self.velocity[i] *= -0.5  # 速度反弹
            self.position = enforce_release_time_constraints(self.
                position)

    # PSO参数
    num_particles = 300
    max_iterations = 200

    # 自适应惯性权重
    w_start = 0.9
    w_end = 0.4

    # 创建粒子群
    particles = [Particle(len(bounds)) for _ in range(num_particles)]

    # 初始化全局最佳
    global_best_position = None
    global_best_fitness = -float("inf")

    # 设置并行池
    pool = multiprocessing.Pool(processes=multiprocessing.cpu_count()
        )

    # 创建历史记录
```

```
    history = {"best_fitness": [], "avg_fitness": []}

    try:
        print(
            f"使用PSO优化5架无人机对抗3枚导弹，粒子数量={num_particles}，最大迭代={max_iterations}"
        )
        print(f"使用{multiprocessing.cpu_count()}个CPU核心进行并行计算")

        # 初始评估
        fitnesses = pool.map(evaluate_position, [p.position for p in particles])

        for i, fitness in enumerate(fitnesses):
            particles[i].fitness = fitness
            particles[i].best_fitness = fitness
            particles[i].best_position = particles[i].position.copy()

            if fitness > global_best_fitness:
                global_best_fitness = fitness
                global_best_position = particles[i].position.copy()

        # PSO主循环
        stagnation_counter = 0
        prev_best = global_best_fitness

        for iteration in range(max_iterations):
            # 计算当前的惯性权重（线性递减）
            w = w_start - (w_start - w_end) * iteration / max_iterations

            # 更新所有粒子
            for p in particles:
                p.update_velocity(global_best_position, w=w)
                p.update_position()

            # 并行评估
            fitnesses = pool.map(evaluate_position, [p.position for p in particles])
```

```
            for i, fitness in enumerate(fitnesses):
                particles[i].fitness = fitness

                # 更新个体最佳
                if fitness > particles[i].best_fitness:
                    particles[i].best_fitness = fitness
                    particles[i].best_position = particles[i].position.copy()

                # 更新全局最佳
                if fitness > global_best_fitness:
                    global_best_fitness = fitness
                    global_best_position = particles[i].position.copy()
                    stagnation_counter = 0

            # 检测停滞并应用扰动
            if abs(global_best_fitness - prev_best) < 0.01:
                stagnation_counter += 1
            else:
                stagnation_counter = 0
                prev_best = global_best_fitness

            if stagnation_counter >= 80:
                print(f"迭代 {iteration}: 检测到停滞, 应用粒子重置")

                # 重置40%的粒子
                reset_count = int(0.4 * num_particles)
                indices = np.argsort([p.fitness for p in particles])[:reset_count]

                for idx in indices:
                    if random.random() < 0.7:
                        # 随机重置
                        particles[idx] = Particle(len(bounds))
                    else:
                        # 基于全局最佳的变异重置
                        new_position = global_best_position + np.random.normal(
```

```
                        0, 0.2, size=len(bounds)
                    ) * np.array([(high - low) for low, high in bounds])
                    # 确保新位置在边界内
                    for j in range(len(new_position)):
                        new_position[j] = min(
                            max(new_position[j], bounds[j][0]), bounds[j][1]
                        )

                    # 创建新粒子并替换
                    new_particle = Particle(len(bounds))
                    new_particle.position = new_position
                    particles[idx] = new_particle

            stagnation_counter = 0

        # 收集历史数据
        current_avg = sum(p.fitness for p in particles) / len(particles)
        history["best_fitness"].append(global_best_fitness)
        history["avg_fitness"].append(current_avg)

        # 打印进度
        if iteration % 5 == 0 or iteration == max_iterations - 1:
            print(
                f"迭代 {iteration}: 最佳适应度 = {global_best_fitness:.4f}, 平均适应度 = {current_avg:.4f}, 缓存数 = {len(fitness_cache)}"
            )

    print("\nPSO优化完成!")
    print(f"最佳适应度值(总干扰时长): {global_best_fitness:.4f}秒")

    # 解码最佳解并生成详细结果
    best_drone_params = []
    for i in range(5):
        drone_params = {
```

```
            "drone_id": i + 1,
            "azimuth": global_best_position[i * 8],
            "speed": global_best_position[i * 8 + 1],
            "releases": [],
        }

        for j in range(3):
            release_time = global_best_position[i * 8 + 2 + j * 2]
            explosion_delay = global_best_position[i * 8 + 3 + j * 2]
            drone_params["releases"].append((release_time, explosion_delay))

        best_drone_params.append(drone_params)

    # 创建最终结果数据
    solution_data = []
    bomb_id = 1

    for drone_idx, drone_param in enumerate(best_drone_params):
        drone_id = drone_param["drone_id"]
        azimuth = drone_param["azimuth"]
        speed = drone_param["speed"]

        # 计算飞行方向向量
        direction_x = np.cos(azimuth)
        direction_y = np.sin(azimuth)
        direction = (direction_x, direction_y, 0)

        # 计算方向角度
        direction_angle = (np.arctan2(direction_y, direction_x) * 180 / np.pi) % 360

        # 计算目标位置
        dir_norm = np.sqrt(direction_x**2 + direction_y**2)
        target_distance = 20000
        drone_target = (
            drone_positions[drone_idx][0]
            + direction_x / dir_norm * target_distance,
```

```
            drone_positions[drone_idx][1]
            + direction_y / dir_norm * target_distance,
            drone_positions[drone_idx][2],
        )

        # 计算每枚烟幕弹的详细信息
        for release_idx, (release_time, explosion_delay) in enumerate(
            drone_param["releases"]
        ):
            # 如果超过20秒，视为不投放
            if release_time > 40:
                continue

            # 计算投放点和起爆点
            release_position = calculate_plane_position(
                drone_positions[drone_idx], direction, speed, release_time
            )

            explosion_position = calculate_bomb_position(
                drone_positions[drone_idx],
                direction,
                speed,
                release_time,
                explosion_delay,
            )

            # 计算对各导弹的干扰时长
            missile_durations = []
            for missile_idx, missile_pos in enumerate(missile_positions):
                try:
                    duration, _ = calculate_shielding_duration(
                        missile_initial_pos=missile_pos,
                        missile_velocity=300,
                        missile_target=(0, 0, 0),
                        plane_initial_pos=drone_positions[drone_idx],
                        plane_target=drone_target,
```

```
                    plane_speed=speed,
                    release_time=release_time,
                    explosion_delay=explosion_delay,
                    target_center=target_center,
                    verbose=False,
                    sample_points=200,  # 提高最终评估的精度
                )
                missile_durations.append(duration)
            except:
                missile_durations.append(0.0)

        # 确保有3个导弹的干扰时长
        while len(missile_durations) < 3:
            missile_durations.append(0.0)

        solution_data.append(
            {
                "无人机编号": f"FY{drone_id}",
                "无人机运动方向": direction_angle,
                "无人机运动速度 (m/s)": speed,
                "烟幕干扰弹编号": bomb_id,
                "烟幕干扰弹投放点的x坐标 (m)":
                    release_position[0],
                "烟幕干扰弹投放点的y坐标 (m)":
                    release_position[1],
                "烟幕干扰弹投放点的z坐标 (m)":
                    release_position[2],
                "烟幕干扰弹起爆点的x坐标 (m)":
                    explosion_position[0],
                "烟幕干扰弹起爆点的y坐标 (m)":
                    explosion_position[1],
                "烟幕干扰弹起爆点的z坐标 (m)":
                    explosion_position[2],
                "针对导弹M1的有效干扰时长 (s)":
                    missile_durations[0],
                "针对导弹M2的有效干扰时长 (s)":
                    missile_durations[1],
                "针对导弹M3的有效干扰时长 (s)":
                    missile_durations[2],
            }
```

```
                )

                bomb_id += 1

        end_time = time.time()
        print(f"\nPSO优化总耗时: {end_time - start_time:.2f}秒")
        df = pd.DataFrame(solution_data)
        df.to_excel("result3.xlsx", index=False)
        print("结果已保存到result3.xlsx")
        return global_best_position, best_drone_params, global_best_fitness

    finally:
        # 关闭并行池
        pool.close()
        pool.join()


# 主函数
if __name__ == "__main__":
    # 设置随机种子

    print("开始粒子群优化五架无人机对抗三枚导弹的策略...")

    try:
        # 运行优化
        best_position, best_params, total_duration = optimize_with_pso_five_vs_three()
        print(f"\n优化成功完成! 最大总干扰时长: {total_duration:.4f}秒")

        # 打印各无人机的最佳参数
        for i, drone in enumerate(best_params):
            print(f"\n无人机FY{i+1}:")
            print(
                f"  运动方向: {np.arctan2(np.sin(drone['azimuth']), np.cos(drone['azimuth'])) * 180 / np.pi:.2f}度"
            )
            print(f"  运动速度: {drone['speed']:.2f} m/s")
            for j, (release_time, explosion_delay) in enumerate(drone
```

```
                ["releases"]):
                if release_time <= 40:  # 有效投放
                    print(
                        f"  烟幕弹{j+1}: 投放时间={release_time:.2f}s, 延迟={explosion_delay:.2f}s"
                    )
                else:
                    print(f"  烟幕弹{j+1}: 不投放")

    except Exception as e:
        print(f"优化过程中出错: {e}")
        import traceback

        traceback.print_exc()
```